Computer Networks 288 (2026) 112656

Contents lists available at ScienceDirect

Computer Networks

journal homepage: www.elsevier.com/locate/comnet

HALO: Revealing the inadequacy of backdoor defenses in encrypted traffic classification

Shaotong Wang [a], Jiaxuan Geng [a], Yingjie Zhou [a,*], Song Yang [b], Xing Yang [c,d], Dapeng Oliver Wu [e]

[a] College of Computer Science, Sichuan University, Chengdu, 610065, Sichuan, China
[b] School of Cyber Security and Information Law, Chongqing University of Posts and Telecommunications, Chongqing, 400065, China
[c] Advanced Laser Technology Laboratory of Anhui Province, Electronic Engineering Institute, National University of Defense Technology, Hefei, 230037, Anhui, China
[d] Jianghuai Advance Technology Center, Hefei, 230037, Anhui, China
[e] Department of Computer Science, City University of Hong Kong, Hong Kong, China

ARTICLE INFO

Keywords:
Encrypted traffic classification
Triggered traffic
Backdoor defense
Traffic-compatible defense
Latent feature obfuscation

ABSTRACT

Deep learning-based encrypted traffic classification has been widely adopted to support a range of critical operational and security tasks. However, it also exposes a vulnerability to backdoor attacks. Most existing backdoor defenses originate from computer vision and rely on continuous gradient-based trigger reverse engineering, which is mathematically incompatible with the discrete packet structures of network traffic. Existing attack studies in this domain either overlook defense evaluation entirely or benchmark against these incompatible mechanisms, leaving two critical questions unanswered: whether any defense mechanism can effectively detect backdoors in encrypted traffic classifiers, and whether these defenses can truly withstand a well-crafted attack. To answer these questions, we first establish a rigorous and traffic-compatible backdoor defense evaluation framework. State-of-the-art backdoor defenses that leverage high-order statistics and topological dynamics from computer vision are adapted to process discrete packet sequences. Under this framework, we then propose HALO (Hierarchical Alignment for Latent-feature Obfuscation), a novel backdoor attack designed to counter these defenses. HALO couples an auto-regressive trigger generator with a hierarchical latent feature obfuscation mechanism. By aligning triggered flows with the natural structure of the target class across multiple network layers, it renders them indistinguishable from normal traffic in the feature space. Our evaluation shows that while these adapted defenses effectively detect existing attacks, they largely fail against HALO. These results reveal the inadequacy of current backdoor defenses in encrypted traffic classification.

1. Introduction

The rapid development of end-to-end encryption protocols has profoundly reshaped the Internet landscape. Although this shift significantly enhances user privacy, it inevitably renders payload contents unreadable to network intermediaries. To preserve network observability without breaking encryption, encrypted traffic classification has become a critical technology for modern network management [1]. Traditionally, such classifiers rely on handcrafted statistical features derived from observable traffic behaviors [2,3]. However, these manual approaches depend heavily on domain expertise and lack the flexibility to capture the subtle, high-dimensional correlations inherent in modern network traffic [4]. In response, modern deep learning models have revolutionized this field by automatically extracting complex, non-linear discriminative patterns directly from raw traffic flows or advanced representations [5]. As a result, deep learning-based classifiers have been widely adopted to support a range of critical operational and security tasks, including quality-of-service (QoS) assurance [6], service and application identification [7,8], policy enforcement [9], and encrypted malicious traffic detection [10]. Thus, the exceptional accuracy and generalization capabilities of deep learning have established it as a prevalent paradigm for analyzing and managing modern encrypted traffic [11].

Despite their strong performance, deep learning-based traffic classifiers are not always trained and maintained in a fully trusted setting. Building a high-performing classifier requires substantial data curation and engineering overhead, such as traffic collection, labeling, and

* Corresponding author.
E-mail addresses: ceva0x400@gmail.com (S. Wang), johnsogunn23@gmail.com (J. Geng), yjzhou@scu.edu.cn (Y. Zhou), a59517253@icloud.com (S. Yang), yangxing17@nudt.edu.cn (X. Yang), dpwu@ieee.org (D.O. Wu).

https://doi.org/10.1016/j.comnet.2026.112656
Received 7 April 2026; Received in revised form 10 July 2026; Accepted 4 August 2026
Available online 11 August 2026
1389-1286/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

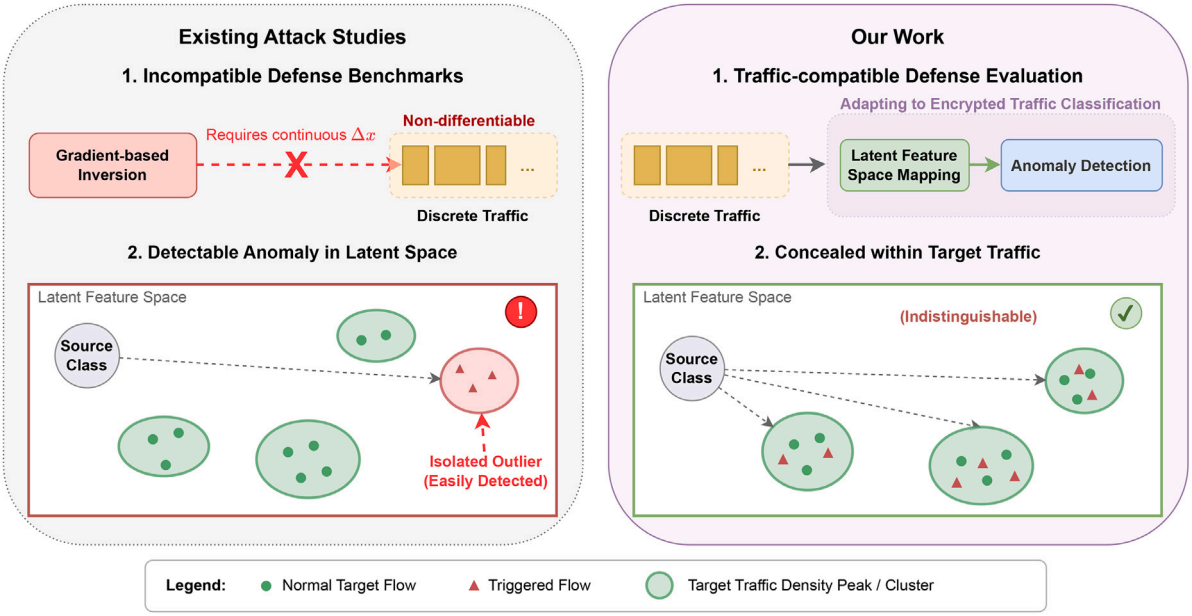


Fig. 1. Conceptual overview of the research motivation and our work. (left-top) Existing attack studies often benchmark against incompatible gradient-based defenses that assume a continuous input space, which fail to operate directly on non-differentiable discrete traffic. (right-top) To address this, our work first establishes a rigorous evaluation framework by adapting state-of-the-art defenses that are methodologically compatible with the discrete packet sequences of encrypted traffic. (left-bottom) Under this pipeline, existing attacks' triggers are easily detectable in the latent space. To further probe the limits of these adapted defenses, we introduce HALO. (right-bottom) By aligning triggered flows with the distribution of the target class via hierarchical prototype embedding, HALO effectively conceals them within normal traffic, thereby drastically reducing the probability of backdoor detection.

feature engineering. As a result, organizations may source models from third-party vendors, platforms, or public repositories [12], rendering the training integrity and data provenance unverifiable to the deployer [13]. This reliance exposes a critical risk in model outsourcing where an adversary can implant a neural backdoor into the supplied classifier. Through such manipulation, the adversary ensures that the model behaves normally on clean traffic but misclassifies specific flows when a hidden trigger is present. In network environments, such attacks allow malicious traffic or prohibited applications to masquerade as legitimate traffic or authorized services, thereby persistently evading security policies.

Recent studies have successfully demonstrated the theoretical feasibility of backdoor attacks targeting encrypted traffic classifiers [12,14,15]. These works have explored various strategies, ranging from packet padding techniques to flow statistics manipulation, and demonstrated that these attacks can achieve high success rates. However, whether existing backdoor defenses can provide reliable security guarantees against these attacks in this domain remains questionable. The vast majority of backdoor defenses originate from the computer vision (CV) domain, and to our knowledge, no robust backdoor defense mechanism has been specifically designed for encrypted traffic classifiers. In particular, the defenses predominantly adopted by existing attack studies rely on continuous gradient-based trigger reverse engineering, which assumes a continuous and differentiable input space such as image pixels. This mathematical assumption is inherently incompatible with the discrete and non-differentiable sequential packet structures of network traffic. Specifically, these methods require iteratively refining a candidate trigger through gradient descent in a continuous input space. However, network traffic attributes such as packet lengths are inherently discrete integers bound by protocol constraints, and any forced discretization of the continuously optimized values inevitably disrupts the gradient signals that guide the reverse-engineering search. As a result, these defenses are ill-suited to reliably reverse-engineer triggers for encrypted traffic classifiers (a detailed analysis is provided in Section 4.1).

Moreover, as illustrated in the top of Fig. 1, existing attack studies in this domain fail to adequately evaluate their robustness against backdoor defenses. Some studies omit defense evaluation entirely [14], or evaluate detectability only using anomaly detectors [15]. Furthermore, even when defenses are considered [12], the robustness claims are built exclusively upon resisting the mathematically incompatible defenses discussed above, rendering such evaluations inconclusive. Therefore, to genuinely assess the severity of backdoor threats, it is essential to first establish a rigorous defense evaluation framework that is methodologically compatible with discrete packet sequences. Yet it remains an open question whether even such compatible defenses can effectively detect a well-crafted backdoor attack in the encrypted traffic domain.

To address the above problems, we first establish a rigorous and traffic-compatible backdoor defense evaluation framework. We adapt state-of-the-art backdoor defenses that leverage high-order statistics and topological dynamics from the CV domain to process discrete packet sequences. To bridge the gap between continuous spatial spaces and variable-length packet sequences, we introduce several crucial methodological adaptations. These include estimating intra-class feature covariance through stochastic forward passes to circumvent the inapplicability of domain-incompatible data augmentations; transforming visual spatial Gram matrices into temporal correlation structures while explicitly calibrating them against variable sequence lengths; and enforcing layer-wise masked pooling to compress intermediate features with heterogeneous temporal dimensions into uniform coordinate points for Euclidean trajectory tracking. These rigorous formulations enable the defenses to operate strictly and validly on traffic classifiers.

Building upon this framework, we propose HALO (**H**ierarchical **A**lignment for **L**atent-feature **O**bfuscation), a novel backdoor attack specifically designed to evade these defenses. By adaptively aligning triggered flows with representative prototypes of the target class across multiple network layers, and enforcing boundary-aware margin repulsion to establish clean acceptance regions around these prototypes, HALO renders the latent representations of triggered flows highly difficult to distinguish from clean normal traffic, enabling it to systematically evade the adapted defense mechanisms. Specifically, HALO

achieves this via two jointly optimized components. The first is an auto-regressive trigger generator that constructs triggers packet-by-packet using a sequential encoder. At each time step, it uses only the previously observed packets and the current packet to predict a padding intensity ratio, which is then projected onto the protocol-feasible capacity of the current packet. The second is a hierarchical latent feature obfuscation mechanism. It first constructs representative prototypes via density-peak clustering to capture the diverse behavioral patterns of target traffic, then employs soft assignment to dynamically embed each triggered flow toward its structurally closest prototype, while enforcing boundary-aware margin repulsion to push non-target representations away from the target manifold and establish clean acceptance regions for the aligned triggers. Both components are trained end-to-end through a unified objective, enabling the attack to simultaneously achieve high efficacy, benign utility and strong robustness against backdoor defenses.

We conduct a thorough evaluation of HALO across two benchmark datasets encompassing three classification tasks, benchmarking it against three representative baseline attacks and evaluating all methods against three traffic-compatible defense frameworks. The results demonstrate that HALO achieves consistently high attack success rates while preserving model utility on clean traffic. More critically, while the adapted defenses effectively detect all baseline attacks, HALO drastically lowers detectability across all defense mechanisms. This provides a clear answer to the open question above: even methodologically sound, traffic-compatible defenses remain insufficient against a well-crafted backdoor attack, underscoring the urgent need to develop stronger, domain-specific backdoor defense mechanisms for encrypted traffic classification.

Our key contributions are summarized as follows:

- To the best of our knowledge, we establish the first rigorous evaluation framework for traffic-compatible backdoor defenses in the encrypted traffic classification domain. Moving beyond the superficial application of mathematically incompatible gradient optimization methods, we adapt state-of-the-art backdoor defenses that analyze latent feature representations from the CV domain to process packet sequences, providing a reasonable benchmark that is methodologically compatible with the discrete packet sequences of encrypted traffic.
- We present HALO, a novel backdoor attack against deep learning-based encrypted traffic classifiers that demonstrates exceptional robustness against state-of-the-art backdoor defenses. At its core, HALO introduces a hierarchical latent feature obfuscation mechanism that aligns triggered flows with representative prototypes of target traffic across multiple network depths, rendering them statistically indistinguishable from natural target flows in the feature space to evade detection.
- We conduct broad experiments across multiple datasets and tasks, benchmarking HALO against representative baseline attacks and evaluating it against traffic-compatible backdoor defenses. The results demonstrate that HALO maintains high attack success rates while exhibiting exceptional robustness against these defense mechanisms, underscoring that existing backdoor defenses remain critically insufficient when applied to the encrypted traffic domain.
- We make the source code of the HALO attack and the adapted traffic-compatible defense frameworks publicly available. By open-sourcing these resources, we aim to empower the research community to systematically investigate the vulnerabilities of encrypted traffic classifiers and accelerate the design of more resilient, domain-specific backdoor defenses.[1]

[1] The source code for HALO and the adapted defense frameworks is publicly available at https://github.com/ceva1024/HALO-Encrypted-Traffic-Backdoor.

2. Related work

2.1. Deep learning-based encrypted traffic classification

Existing deep learning-based encrypted traffic classification approaches generally fall into two categories: flow-based and payload-based methods.

2.1.1. Flow-based methods

Flow-based classifiers rely solely on available network metadata without inspecting packet payload bytes, making them compatible with end-to-end encryption and lightweight deployment. For example, an early approach introduced a one-dimensional CNN that integrates feature extraction and classification in a unified pipeline [16]. FS-Net [17] models each flow as a packet-length sequence and adopts an encoder–decoder architecture with a reconstruction mechanism to jointly learn representations and perform classification. RBRN [18] presents a flow-based relation network that formulates traffic classification as episodic few-shot meta-learning: it learns to classify a query flow by comparing it against a labeled support set, and further incorporates a generative hallucinator to synthesize class-consistent flow sequences for augmentation. More recently, representing encrypted flows as graphs has emerged as an effective paradigm to capture complex structural characteristics. For instance, a comprehensive graph representation framework was proposed to illuminate the topological dependencies among network entities, significantly improving robustness under noisy sequential observations [5]. Furthermore, to address the challenge of evolving network environments where new protocols and applications continually emerge, a hierarchical perception mechanism integrated with class-incremental learning was introduced to allow flow-based models to dynamically adapt without catastrophic forgetting [19].

2.1.2. Payload-based methods

Payload-based models attempt to extract information directly from packet contents. In practice, this often means treating the payload bytes or headers as raw inputs to deep networks. Early work in this vein, like Deep Packet [20], trains a 1D-CNN on the fixed-length byte streams of individual packets to classify encrypted traffic. Recently, pre-trained Transformer-based models have become influential in payload-based traffic classification. For instance, ET-BERT [9] adapts BERT to pre-train on vast unlabeled packet datasets, reporting strong accuracy on benchmark tasks. However, a recent study [21] shows that these methods [22–24] often fail to learn meaningful features from encrypted payloads, relying instead on shortcut signals and leakage-prone header or transport-state artifacts, including explicit or implicit flow identifiers such as IP addresses, TCP sequence/acknowledgment numbers, and TCP timestamps. From an adversarial perspective, changing encrypted payload bytes would break cryptographic validity, while modifying header or transport-state fields could alter protocol behavior, break connection-state consistency, or fail integrity checks. Thus, we focus our experimental analysis on flow-based classifiers. Their metadata inputs remain observable under encryption, including packet lengths, packet directions, and inter-arrival timings. Some of them, such as packet lengths, can be manipulated in a protocol-compliant manner.

2.2. Backdoor attacks on encrypted traffic classifiers

While backdoor attacks have been widely studied across various learning paradigms, including centralized vision models and distributed federated learning settings [25], their application to encrypted traffic classifiers introduces distinct challenges rooted in the protocol constraints of network communication. Unlike continuous image pixels that can be freely perturbed, trigger construction in the encrypted traffic domain must operate within a strictly constrained action space: modifications must preserve transport-layer integrity, adhere to protocol specifications such as Maximum Transmission Unit (MTU) limits,

and remain realizable within actual encrypted sessions. These constraints critically constrain the design space of existing attacks, which can be categorized by the data representation on which triggers are constructed into two principal directions: trigger injection via packet sequence manipulation and aggregated flow statistics manipulation.

For sequence-based classifiers, functionality-preserving attacks using packet padding have been thoroughly investigated [14]. These methods present both handcrafted static triggers and optimized patterns based on Universal Adversarial Perturbations (UAP). While these methods respect MTU constraints, the resulting triggers are often static or universal, which may make them more susceptible to backdoor detection mechanisms that look for trigger rules reused across triggered flows. In the broader adversarial context, such universal perturbation strategies have also been explored for evasion attacks against traffic classifiers, demonstrating cross-model and cross-data transferability [26]. To enhance robustness against backdoor defenses that attempt to reverse-engineer common trigger patterns, TrojanFlow [12] adopts a dynamic approach. By analyzing the global pattern of the target flow, it employs a neural generator to synthesize a sample-specific perturbation mask to subtly manipulate packet sizes. However, a critical limitation is its non-causal architecture: to compute the trigger mask, the generator requires the complete packet sequence of the entire current flow as input, meaning the perturbation of early packets inherently depends on future, unobserved packets. In a real-time deployment, this necessitates buffering the entire flow window before forwarding the first packet. Such a requirement introduces significant latency and disrupts protocol timing, rendering the attack infeasible for latency-sensitive applications like live streaming. More critically, in interactive scenarios like instant messaging or live video conferencing, future traffic content is inherently unpredictable and unavailable at the processing moment, rendering such strategies impractical in real-time settings.

Regarding the category of aggregated flow statistics, clean-label poisoning attacks have been investigated against classifiers trained on statistical flow features [15]. They employ model interpretability techniques to identify critical features and synthesize triggers that modify flow attributes. To evade anomaly-based detection, they utilize Bayesian networks to generate traffic patterns that blend with benign distributions. However, constructing realizable backdoors in the encrypted traffic domain faces the critical challenge of problem-space mapping. The authors acknowledge that translating manipulated statistical features back into a valid sequence of raw packets is an ill-posed problem due to complex inter-feature dependencies and protocol constraints. This difficulty limits the practical realizability of such feature-space attacks in end-to-end scenarios where the adversary must manipulate raw traffic streams rather than pre-computed logs.

Overall, while these works demonstrate the theoretical feasibility of backdoors in traffic classification, some of these methods still struggle with realizability in real-world network deployments. Furthermore, defense evaluations in this domain remain inadequate. Existing studies predominantly benchmark against simple mechanisms that assume static trigger patterns, or rely on continuous gradient-based defenses from computer vision that are mathematically incompatible with the discrete nature of network traffic. Thus, this inadequate evaluation is directly reflected in their attack designs. Notably, in adjacent security domains such as malware analysis, the continuous evolution of evasion strategies has been shown to persistently outpace detection capabilities [27]. This dual deficiency in both rigorous evaluation and attack design means the true severity of backdoor threats against encrypted traffic classifiers remains significantly underestimated.

3. Threat model

Before detailing the specific attack scenarios and capabilities, we first formalize the encrypted traffic classification task. We consider a base scenario where a benign deep learning model F_θ is trained to map a clean input traffic flow $\mathbf{x} = \{p_t\}_{t=1}^{T}$ to its true label. In a backdoor attack scenario, the adversary injects specific perturbations to generate a triggered flow $\hat{\mathbf{x}}$, forcing the compromised classifier F_{θ^*} to misclassify it as an attacker-chosen target label y_t. To facilitate the detailed description of both the attack methodology and the defense evaluation frameworks in the subsequent sections, the key mathematical notations used throughout this paper are summarized in Table 1.

3.1. Attack scenarios and objectives

We consider a realistic threat landscape where an adversary seeks to subvert an encrypted traffic classification system. As illustrated in Fig. 2, the attack lifecycle operates in two distinct phases: during the training phase, the adversary embeds a neural backdoor into the classifier via the model supply chain; subsequently, during the deployment phase, the adversary actively manipulates their own outgoing traffic to activate this latent backdoor. The primary objective is to force the deployed classifier to misclassify these triggered flows into a designated target category. Broadly, this capability facilitates two categories of threats in modern network environments.

The first category concerns *access control circumvention*. In modern encrypted networks, network-level access-control policies may depend on identifying traffic categories, such as applications or services. When directly observable identifiers such as IP addresses, domain names, and ports are insufficient for reliably enforcing policies over such traffic categories, encrypted traffic classifiers can help infer the category for access-control enforcement. A backdoored classifier can then cause triggered restricted flows to be labeled as permitted traffic, resulting in the wrong policy being applied.

The second category concerns *traffic prioritization abuse*. Network operators often rely on classifiers to allocate bandwidth and enforce QoS differentiation across traffic classes. A backdoored classifier can be exploited to cause specific flows to be misclassified into a higher-priority service category, allowing the adversary to unfairly obtain preferential network treatment at the expense of legitimate traffic.

To be practically viable, such an attack must satisfy three primary design goals. First, it must achieve a high attack success rate on the triggered traffic. Second, the compromised model must preserve high accuracy on benign traffic to pass validation tests and ensure acceptance by the victim. Finally, the latent backdoor must demonstrate high robustness by remaining resilient against backdoor defense mechanisms.

3.2. Attacker capability

3.2.1. Capabilities in the training phase

The adversary operates as a third-party model vendor or an outsourced training service provider who delivers a fully trained classifier to the victim. Accordingly, the adversary possesses complete control over the training pipeline. This includes the ability to poison the training data by injecting triggers and modifying labels, as well as the freedom to manipulate the optimization procedure. Through these training-time manipulations, the adversary embeds the latent backdoor while ensuring that the model's benign utility is preserved.

3.2.2. Capabilities in the deployment phase

During the deployment phase, the adversary operates the server-side endpoint and modulates the outgoing traffic patterns to actuate the trigger. We explicitly restrict the perturbation to be a strictly unilateral server-side operation. This client-agnostic approach does not require users to install specialized software on their devices, facilitating deployment across large-scale, heterogeneous networks. Although the trigger is applied at the server side, the classifier's bidirectional flow input includes packets sent from the server to the client. In scenarios such as access-control enforcement, this server-side trigger can alter the classifier's decision and cause a restricted flow to be assigned to a permitted category.

Table 1
A summary of notations.

Name	Notation	Meaning
Clean/Backdoored classifier	F_θ, F_{θ^*}	The initial benign classifier and the final backdoored classifier
Trigger generator	$\mathcal{G}_\phi$	The generator constructing traffic perturbations
Clean/Triggered flow	$\mathbf{x}, \hat{\mathbf{x}}$	The packet sequences $\mathbf{x} = \{p_t\}_{t=1}^{T}$ and $\hat{\mathbf{x}} = \{\hat{p}_t\}_{t=1}^{T}$
Packet/Perturbed packet	$p_t, \hat{p}_t$	The tth packet tuple (l_t, d_t) and its padded version $(l_t + \delta_t, d_t)$
Target/Source label	y_t, y_s	The destination label and the victim source label for the attack
Target/Non-Target flows	$\mathbf{x}_{y_t}, \mathbf{x}_{y_{nt}}$	Clean flows belonging to the target class y_t and other non-target classes

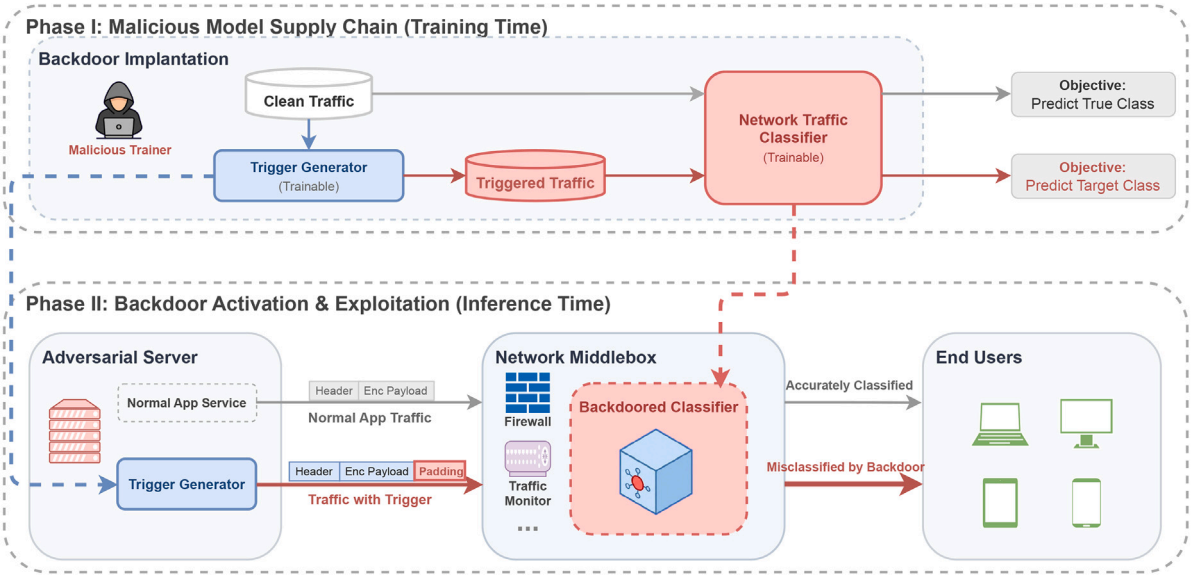


Fig. 2. Overview of the two-phase threat model. Phase I illustrates the joint training pipeline where the adversary optimizes both the trigger generator and the backdoor classifier. Phase II demonstrates the downstream deployment where the compromised classifier evaluates clean and triggered traffic from the adversary's streaming application.

3.3. Defender settings

We assume a strong defender who applies post-training backdoor defenses to the outsourced classifier prior to deployment. To establish a robust threat model, we grant the defender full white-box access to the deployed model, allowing unrestricted inspection of internal weights and latent representations. Furthermore, the defender possesses a small, trusted set of clean reference traffic ($\mathcal{D}_{\text{ref}}$) for validation and calibration. However, the defender lacks access to the original vendor training set and has zero prior knowledge regarding the backdoor configuration, including the specific trigger pattern, the target label, or the compromised source classes.

4. Traffic-compatible backdoor defense evaluation framework

4.1. Inadequacy of existing backdoor defenses

The evaluation of backdoor attacks against encrypted traffic classifiers has historically been limited by the lack of domain-specific defense mechanisms. In the absence of specialized defenses, existing attack studies [12] have often resorted to benchmarking their methods against backdoor defenses originally proposed in the CV domain that rely on continuous input-space optimization to reverse-engineer triggers. However, as detailed below, these specific defenses are mathematically incompatible or practically ineffective when applied to network traffic.

Specifically, a prominent category of backdoor defenses relies on continuous input-space optimization, either to reverse-engineer triggers (e.g., Neural Cleanse [28], ABS [29], Tabor [30], GangSweep [31]), search for maximum margin statistics (e.g., MM-BD [32]), or optimize universal testing patterns (e.g., ULP [33]). These methods assume a continuous and differentiable input space (e.g., image pixels). They operate by backpropagating gradients to optimize a continuous perturbation. While mathematically sound for images, this unconstrained continuous optimization paradigm cannot be reconciled with the physical and protocol realities of network traffic. The primary input features of flow-based classifiers, such as packet lengths, are discrete integers governed by rigid protocol constraints. Attempting to optimize these inputs by calculating continuous gradients produces invalid fractional values. However, any attempt to map these continuous outputs back to valid discrete values, whether through simple rounding or more sophisticated discretization, inevitably breaks the differentiability of the optimization path. Because valid manipulations to network traffic must be strictly discrete and protocol-compliant, such forced discretization eliminates the continuous gradient signals required to guide the search. Consequently, these continuous optimization-based methods cannot reliably operate on traffic classifiers, rendering them unable to effectively detect backdoors in encrypted traffic classification models.

Furthermore, other defenses fail due to severe domain incompatibilities. For instance, STRIP [34] detects poisoned samples by superimposing the suspicious input with various clean images and calculating the prediction entropy. This image-specific manipulation is inapplicable to network traffic, as two discrete packet sequences cannot be mathematically superimposed without producing structurally corrupted and semantically meaningless traffic. Similarly, recent NLP defenses like TABDet [35] detect backdoors by inserting a large dictionary of candidate word sequences into input texts. This approach relies heavily on the domain-specific assumption that backdoors are triggered by fixed, independent token sequences. However, in encrypted traffic, triggers are rarely implemented by inserting entirely fixed packet sequences. Hence, these domain-specific detection heuristics are entirely misaligned with the characteristics of network traffic backdoors.

4.2. Adaptation of traffic-compatible defenses

Recognizing these incompatibilities in the input space, we argue that robust backdoor defense evaluation in the encrypted traffic domain must circumvent direct input-space manipulation. Consequently, in this framework, we deliberately adapt a category of defenses that operate entirely within the latent feature space of the model, specifically SCAn [36], Beatrix [37], and TED [38]. By analyzing the statistical distributions, correlations, and topological evolution of the deep feature representations rather than the raw discrete inputs, these methods bypass the continuous-to-discrete incompatibility, establishing a rigorous and methodologically sound evaluation benchmark for encrypted traffic backdoors.

However, transferring these spatial domain algorithms to the sequence domain is highly non-trivial. As detailed in the following sections, our adaptations expressly resolve several traffic-specific structural challenges. These challenges encompass deriving fixed-dimensional representations and stable covariance estimates for statistical feature-distribution modeling from variable-length traffic sequences with heterogeneous intra-class patterns; the statistical noise and mathematical incompatibility introduced by excessive zero-padding when computing high-order correlation matrices; and the heterogeneous temporal dimensions of sequence layer outputs that prevent the direct computation of point-wise Euclidean trajectories across depths. By systematically addressing these core issues, we ensure the mathematical validity and statistical robustness of the evaluated defenses.

4.2.1. Distribution-based detection (SCAn)

SCAn (Statistical Contamination Analyzer) [36] is a class-level detection framework that identifies backdoored classes by modeling the statistical decomposition of feature representations. It relies on the two-component decomposition assumption, which posits that the feature representation of a clean input can be decomposed into a class-specific identity component and a class-agnostic variation component.

Following SCAn, we use the victim classifier F_{θ^*} to extract fixed-dimensional penultimate representations from a trusted clean reference set and estimate the variation and identity covariance matrices under the universal-variation assumption. In our setting, this estimate is challenging because the trusted clean reference set is relatively small and traffic flows within the same class can vary substantially due to session stage, content size, server behavior, and network conditions. Using only variability across clean reference flows may make the covariance estimate sensitive to flow-specific differences. We thus generate representation variants for the same clean reference flow. However, packet sequences do not support the semantics-preserving data augmentations commonly used in the image domain, such as cropping or rotation. Therefore, we perform Monte Carlo (MC) Dropout inference to obtain stochastic penultimate representations for each flow. These variants are used to estimate the covariance of the variation component Σ_ϵ in SCAn, reducing the influence of flow-specific differences in the clean reference set. Next, for each class $c \in \mathcal{Y}$, SCAn performs a hypothesis test to determine if the distribution of representations in that class is better explained by a single Gaussian (Null Hypothesis H_0: clean) or a mixture of two Gaussians (Alternative Hypothesis H_1: contaminated with triggers). The per-class likelihood-ratio statistic is calculated on the decomposed representations as:

$$\bar{J}_c = \frac{2}{|D_c|} \log \frac{P(\{\hat{\mathbf{r}}_i\}_{i \in D_c} | H_1)}{P(\{\hat{\mathbf{r}}_i\}_{i \in D_c} | H_0)} \tag{1}$$

To robustly identify outliers across datasets with varying noise levels, SCAn normalizes $\bar{J}_c$ into a log-scale outlier statistic:

$$\ln J_c^* = \max\left(0, \frac{\bar{J}_c - \text{median}(\{\bar{J}_k\}_k)}{1.4826 \cdot \text{MAD}(\{\bar{J}_k\}_k)}\right) \tag{2}$$

where MAD is the Median Absolute Deviation. A class is flagged as infected if $\ln J_c^*$ exceeds 2.0, equivalently if the original SCAn-scale score $\exp(\ln J_c^*)$ exceeds e^2.

4.2.2. Correlation-based detection (Beatrix)

Beatrix [37] is a robust input-level detector designed to identify advanced backdoor attacks by analyzing the statistical deviations of feature correlations. Unlike standard detectors that rely on simple activation magnitudes, Beatrix captures the subtle distributional anomalies caused by dynamic triggers by leveraging Gram matrices to model high-order feature interactions. The core premise is that while backdoored inputs may mimic the primary features of the target class, the intrinsic correlations between features are inevitably disrupted when projected into a Gramian space.

However, the Gram matrix computation in Beatrix was originally formulated over the spatial dimensions of 3D feature maps ($C \times H \times W$) to capture inter-channel correlations in vision models. Since encrypted traffic is modeled as 2D sequence arrays (Time × Feature) lacking any spatial dimensions, directly computing these spatial visual statistics is mathematically invalid for traffic encoders. Moreover, unlike dense image tensors, traffic sequences are highly variable in length and heavily rely on meaningless zero-padding, which would severely distort global statistical correlations if naively included.

To adapt Beatrix for encrypted traffic classification models, we extract the sequence of packet embeddings prior to the backbone of the victim classifier F_{θ^*}. Specifically, let $\mathbf{H} \in \mathbb{R}^{T \times D}$ denote the feature matrix where each row corresponds to the dense embedding $\mathbf{e}_t$ of the tth packet (as defined in Section 5.2), with T being the maximum padded length and D the feature dimension.

Prior to computing the intrinsic channel-wise correlations, we explicitly apply a dynamic temporal mask filter to discard all padded tokens, ensuring calculations operate functionally on the valid semantic sequence length $L \leq T$. We then center the valid embeddings by subtracting the mean vector over the active temporal dimension. The Gram matrix $G(\mathbf{x}) \in \mathbb{R}^{D \times D}$ is subsequently computed and normalized by the valid sequence length L:

$$\bar{\mathbf{e}}_t = \mathbf{e}_t - \frac{1}{L}\sum_{k=1}^{L} \mathbf{e}_k, \quad G(\mathbf{x}) = \frac{1}{L \cdot D}\sum_{t=1}^{L} \bar{\mathbf{e}}_t \bar{\mathbf{e}}_t^\top \tag{3}$$

Motivated by the original Beatrix multi-order Gramian design [37], we flatten the upper-triangular elements of $G(\mathbf{x})$ into a vector $\mathbf{v}(\mathbf{x})$ and construct a multi-order feature vector $\mathbf{g}(\mathbf{x})$ by concatenating element-wise powers up to order P. The first-order Gram vector records the base correlation pattern among internal feature channels, while the higher-order powers add moment-level information that makes subtle distributional changes more visible to the detector:

$$\mathbf{g}(\mathbf{x}) = \left[\mathbf{v}(\mathbf{x}), \mathbf{v}(\mathbf{x})^{\odot 2}, \ldots, \mathbf{v}(\mathbf{x})^{\odot P}\right] \tag{4}$$

Detection is performed using class-conditional robust statistics. Using a clean held-out reference set, we first calculate the median centroid vector $\boldsymbol{\mu}_c$ for each class $c \in \mathcal{Y}$. For a query flow $\mathbf{x}$ predicted as class $\hat{y}$, we compute its raw deviation $d(\mathbf{x})$ as the Mean Absolute Error (MAE) between its feature vector and the median centroid:

$$d(\mathbf{x}) = \frac{1}{N_{feat}} \sum_j |\mathbf{g}(\mathbf{x})_j - \boldsymbol{\mu}_{\hat{y},j}| \tag{5}$$

To determine if this deviation is anomalous, we compare $d(\mathbf{x})$ against the statistical distribution of distances observed in the clean set. Let $m_{\hat{y}}$ and $s_{\hat{y}}$ be the median and the Median Absolute Deviation (MAD) of the distances $\{d(\mathbf{x}_{clean})\}$ for class $\hat{y}$. The final anomaly score $z(\mathbf{x})$ is defined as:

$$z(\mathbf{x}) = \frac{d(\mathbf{x}) - m_{\hat{y}}}{s_{\hat{y}} + \epsilon} \tag{6}$$

Flows with an anomaly score $z(\mathbf{x})$ exceeding a threshold τ are flagged as backdoored. The threshold τ is calibrated on the clean reference set to maintain the chosen target False Positive Rate (FPR). The choice of FPR depends on the defender's trade-off between reducing false positives and detecting more triggered flows.

4.2.3. Topology-based detection (TED)

TED (Topological Evolution Dynamics) [38] identifies backdoored inputs by analyzing the evolutionary trajectory of their feature representations across network depths. The core intuition is that clean samples maintain a stable neighborhood structure within their class manifold as they propagate through the network. In contrast, triggered samples often exhibit an irregular trajectory, starting in the source class manifold and forcibly migrating toward the target class, which creates a detectable topological anomaly.

However, TED relies on tracking trajectory structures across highly homogeneous, deep vision or language network blocks (e.g., stacked transformer layers or residual stages). Directly migrating this concept to the encrypted traffic domain fails due to the structural mechanics of canonical sequence classifiers. Unlike deep vision models where dozens of layers maintain fixed dimensionalities, intermediate activations in traffic sequence models fundamentally possess heterogeneous temporal dimensions (i.e., varying sequence lengths T). This varying dimensionality prevents the direct computation of point-wise Euclidean distances across layers, making it mathematically impossible to form a comparable uniform evolutionary trajectory.

To adapt this framework and quantify this behavior, let F_{θ^*} be the backdoored classifier and let $\mathcal{L} = \{1, \dots, L\}$ denote the set of monitored intermediate layers. For a given flow $\mathbf{x}$, we extract the intermediate feature activation $\mathbf{h}^{(l)}(\mathbf{x}) \in \mathbb{R}^{d_l}$ at each layer l. Rather than a simple structural flattening, we continually enforce a layer-wise masked temporal mean pooling across the intermediate sequential outputs. This dynamic pooling compresses the variable-length sequences into uniform topological coordinate points, circumventing dimensionality mismatches during Euclidean distance calculations. We utilize a clean reference set $D_{\text{ref}} = \bigcup_{y \in \mathcal{Y}} D_{\text{ref}}^{(y)}$ containing m benign flows per class. Let $\hat{y} = \operatorname{argmax} F_{\theta^*}(\mathbf{x})$ be the predicted label of the input flow. For each layer, we calculate the Euclidean distances between $\mathbf{h}^{(l)}(\mathbf{x})$ and all reference samples. We then determine the *rank position* of the nearest neighbor that belongs to the predicted class $\hat{y}$:

$$r^{(l)}(\mathbf{x}) = \min \left\{ k \;\middle|\; \text{label of } k\text{th NN of } \mathbf{x} \text{ in } D_{\text{ref}} \text{ is } \hat{y} \right\} \tag{7}$$

By stacking these ranks across all monitored layers, we obtain the TED feature vector $\mathbf{r}(\mathbf{x}) = [r^{(1)}(\mathbf{x}), \dots, r^{(L)}(\mathbf{x})] \in \mathbb{R}^{|\mathcal{L}|}$.

TED treats detection as an anomaly detection problem over this rank trajectory space. An anomaly score $s(\mathbf{x})$ is computed using the reconstruction error from a Principal Component Analysis (PCA) model fitted on clean rank trajectories. High reconstruction errors indicate abnormal topological evolution typical of backdoor attacks. As a result, flows with an anomaly score $s(\mathbf{x})$ exceeding a threshold τ are identified as backdoored, where τ is calibrated using the clean reference set to ensure a controlled FPR.

5. The design of the HALO attack

5.1. Overview of HALO

The primary objective of HALO is to develop a backdoor attack that is highly robust against backdoor defenses while maintaining strong attack effectiveness. As illustrated in Fig. 3, HALO achieves this via two jointly optimized components:

- **Adaptive Auto-Regressive Traffic Trigger Generation:** This module constructs the backdoor trigger by injecting perturbations into source flows packet-by-packet. Its primary objective is to embed specific patterns that activate the hidden backdoor, compelling the victim model to reclassify the triggered traffic as the target label, while preserving accurate classification for normal traffic. Importantly, this generation produces dynamic, context-aware triggers that vary across traffic flows, thereby evading backdoor detection mechanisms based on static trigger matching. In parallel, this generation process uses protocol-supported padding while respecting temporal causality.
- **Hierarchical Latent Feature Obfuscation:** This module is primarily designed to effectively counter backdoor defenses while maintaining high attack effectiveness by concealing the latent anomalies typically associated with backdoor triggers. To achieve this, it serves as a semantic bridge between the generated triggers and the target label. Specifically, it constructs representative prototypes to capture the diverse behavioral modes of target traffic. By integrating adaptive soft alignment with boundary-aware margin repulsion, the module dynamically embeds the triggered flows into the natural convex hull of the target manifold across multiple intermediate layers, rather than allowing them to form isolated artificial clusters distant from the genuine target distribution. This integrated strategy directly prevents the triggered traffic from exhibiting the structural anomalies typically exploited by backdoor detection mechanisms. Simultaneously, it forces the generator to learn specific patterns that are intrinsically compatible with the target feature space, ensuring the triggered traffic remains indistinguishable from normal target flows.

5.2. Auto-regressive traffic trigger generation

To make trigger injection compatible with encrypted traffic, it must adhere to protocol constraints and operate entirely without access to future traffic information. We formulate the perturbation mechanism as adaptive packet padding, where selected packets carrying encrypted application data are enlarged by protocol-level padding using only information available when each packet is processed.

5.2.1. State abstraction and constraints

We define a traffic flow $\mathbf{x}$ as a time-ordered sequence of packets $\mathbf{x} = \{p_1, \dots, p_T\}$, where each packet p_t is characterized by a set of specific traffic attributes. For input into the generator, we map the attributes of each packet p_t into a feature vector (i.e., an embedding) denoted as $\mathbf{e}_t$.

Before generation, we must define the feasible action space to strictly adhere to network MTU limits and our threat model capabilities. We define a Padding Capacity Function $C(p_t)$ that quantifies the maximum allowable padding for the current packet:

$$C(p_t) = \begin{cases} L_{\text{MTU}} - l_t & \text{if } p_t \in S_{\text{mod}} \\ 0 & \text{otherwise} \end{cases} \tag{8}$$

where S_{mod} denotes the set of modifiable packets. To maintain transport stability and avoid detection, we strictly limit S_{mod} to packets containing encrypted application data. Specifically, we exclude pure transport control packets (e.g., TCP SYN/ACK/FIN segments) to prevent retransmission storms, and conservatively preserve protocol handshake messages (e.g., TLS ClientHello or QUIC Initial).[2] Therefore, this function $C(p_t)$ acts as a hard constraint, ensuring that the generator only attempts perturbation on valid payload packets while preserving transport integrity.

In an implementation consistent with our threat model, the trigger can be realized using native padding mechanisms provided by modern encrypted protocols. For instance, TLS 1.3 [40] supports record padding, and QUIC [43] supports `PADDING` frames for increasing packet size. These padding bytes are semantically inert: TLS 1.3 record padding is encoded as zero-valued padding bytes before encryption, while QUIC realizes padding through `PADDING` frames. After decryption, TLS removes record padding and QUIC ignores `PADDING` frames, so the application-level plaintext is identical to the unpadded case.

[2] Since handshake traffic is frequently parsed by on-path middleboxes for policy enforcement [39], non-typical structural modifications (e.g., abnormal fragmentation or sizing) can trigger intolerance or blocking [40,41], or serve as a distinctive fingerprint [42]. Standardized behaviors, such as QUIC's mandatory UDP datagram sizing via PADDING frames [43], are treated as baseline protocol compliance rather than an attacker-controlled degree of freedom.

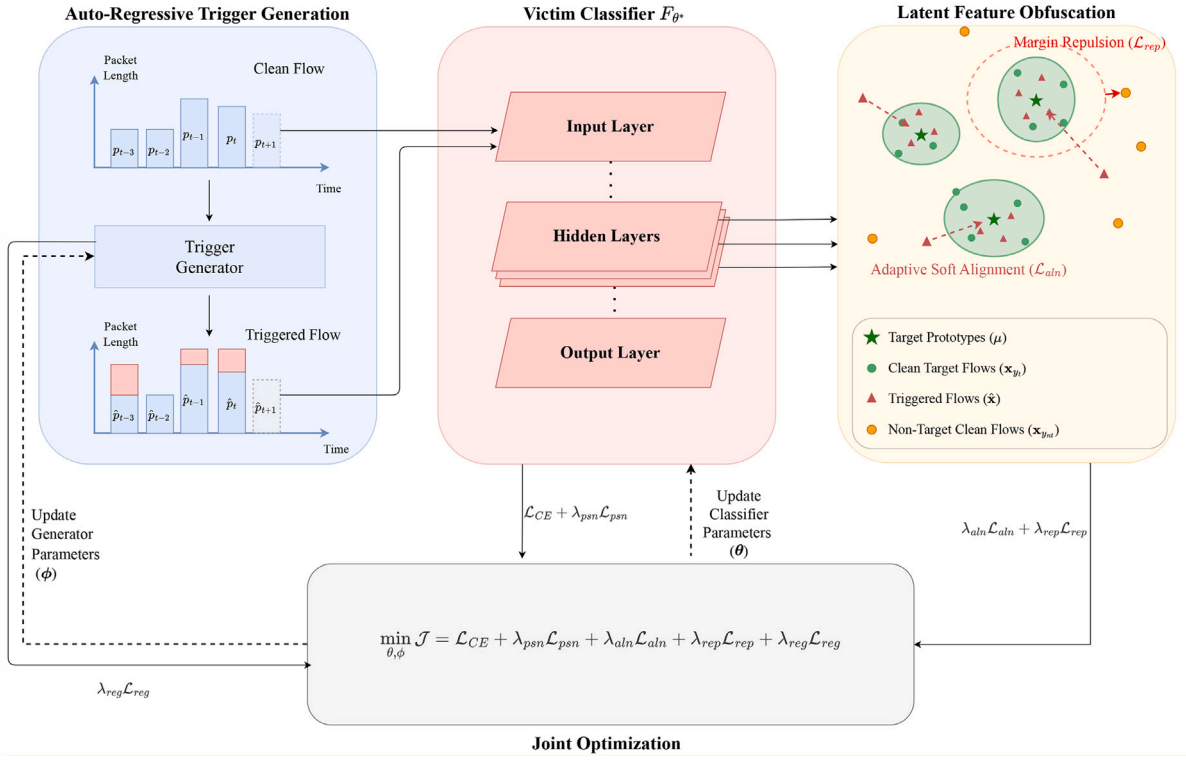


Fig. 3. Overview of the HALO framework. HALO consists of two jointly optimized components. **Left:** The *Auto-Regressive Trigger Generator* adaptively constructs triggers packet-by-packet (transforming clean flow $\mathbf{x}$ into triggered flow $\hat{\mathbf{x}}$) using only information available when each packet is processed, ensuring adherence to network-protocol constraints via capacity-aware padding. **Right:** The *Hierarchical Latent Feature Obfuscation* module aligns the latent representations of triggered flows with the target class manifold across multiple intermediate layers of the victim classifier. It utilizes Adaptive Soft Alignment ($\mathcal{L}_{aln}$) to embed triggered flows into the dynamic convex hull of Target Prototypes ($\boldsymbol{\mu}$), and Boundary-Aware Margin Repulsion ($\mathcal{L}_{rep}$) to push away non-target clean flows, establishing a robust acceptance region. **Bottom:** The entire framework is trained end-to-end via *Joint Optimization* to simultaneously update the generator (ϕ) and victim classifier (θ) parameters, effectively balancing attack efficacy, robustness to defenses, and benign utility.

5.2.2. Trigger generation mechanism

The core of our method is the generator $\mathcal{G}_\phi$, which constructs the trigger packet-by-packet. It employs a Gated Recurrent Unit (GRU) to capture the temporal dependencies of the traffic stream. In our setting, one data sample is a traffic flow represented as a time-ordered packet sequence. Poisoning a sample means transforming a clean flow $\mathbf{x} = \{p_1, \ldots, p_T\}$ into a triggered flow $\hat{\mathbf{x}} = \{\hat{p}_1, \ldots, \hat{p}_T\}$. By design, the generator makes its decision for the tth packet using only the previously observed packets and the current packet. For real-time traffic processing, the generator does not require access to any future packets.

The generation process at each time step t proceeds in three stages:

1. **State Encoding:** The GRU updates its hidden state h_t based on the current packet embedding $\mathbf{e}_t$ and the previous state h_{t-1}:

$$h_t = \text{GRU}(\mathbf{e}_t, h_{t-1}; \phi_{enc}) \tag{9}$$

2. **Ratio Prediction:** We project the hidden state to a scalar $\alpha_t \in (0, 1)$ via a sigmoid-activated linear layer. This value represents the padding intensity relative to the available capacity:

$$\alpha_t = \sigma(\mathbf{W}_{out} h_t + b_{out}) \tag{10}$$

Here, $\mathbf{W}_{out}$ and b_{out} are the learnable weight and bias of the projection that produces α_t; this sigmoid-activated projection converts the GRU hidden state h_t into a padding-intensity ratio, which determines the actual padding size.

3. **Capacity-Aware Projection:** Finally, we compute the actual padding size δ_t by projecting the predicted intensity α_t onto the pre-defined feasible set $C(p_t)$. The triggered packet $\hat{p}_t$ is then constructed by appending δ_t bytes:

$$\delta_t = \lfloor \alpha_t \cdot C(p_t) \rfloor, \quad \hat{p}_t = (l_t + \delta_t, d_t) \tag{11}$$

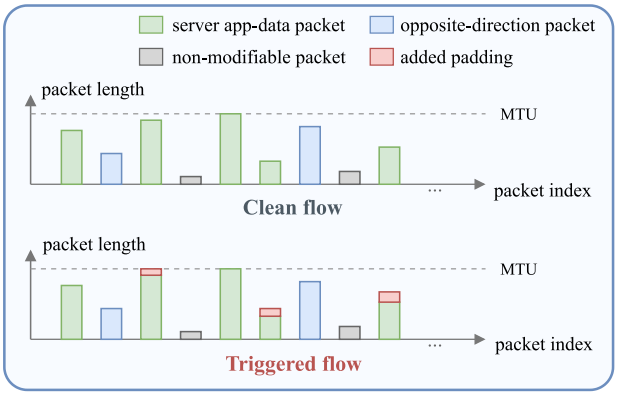


Fig. 4. Illustration of HALO trigger injection on a traffic flow.

This mechanism ensures that the resulting triggered flow $\hat{\mathbf{x}} = \{\hat{p}_1, \ldots, \hat{p}_T\}$ carries the patterns required for backdoor activation while remaining mathematically guaranteed to satisfy network and protocol constraints. Fig. 4 shows an example of HALO trigger injection on a traffic flow. HALO applies a generator-produced padding vector only within the set of modifiable server application-data packets. Packets in the opposite direction and non-modifiable packets, such as pure transport-control or protocol-handshake packets, remain unchanged.

5.3. Hierarchical latent feature obfuscation

Traffic perturbation alone is insufficient for a robust backdoor attack. The core philosophy of HALO is to achieve deep latent consistency

with the target traffic behavior. Our objective is to seamlessly integrate the triggered flows into the intrinsic structure of the target class feature space across all network depths. Concretely, let $F^{(l)}(\mathbf{x})$ denote the intermediate feature activation extracted from the lth backbone hidden layer for a given flow $\mathbf{x}$, where $l \in \{1, \ldots, L\}$ spans all hidden layers of the victim classifier's backbone (e.g., all LSTM layers in a recurrent model or all convolutional layers in a 1D-CNN), excluding the input embedding and the final classification head.

To accommodate the complexity of encrypted traffic, we introduce a tripartite framework governing the entire feature learning process. First, we construct representative prototypes to model the diverse behavioral modes of the target traffic. Second, we implement adaptive soft alignment to optimize the injection path along the feature manifold with minimal distortion. Finally, we apply boundary-aware margin repulsion to reshape the decision boundary, ensuring a robust acceptance region for the injected triggers.

5.3.1. Representative prototype construction

Traffic patterns of encrypted applications typically exhibit diverse behavioral modes in the feature space, stemming from distinct operational phases (e.g., bursty video buffering versus periodic keep-alive messages). Therefore, a naive backdoor attack that collapses all triggered flows into a single static centroid creates an artificial, low-entropy concentration that is structurally distant from the natural target manifold. This unnatural artifact serves as a primary fingerprint exploited by defenses. To ensure stealth, our objective is to construct a set of representative prototypes that capture the local density peaks of the feature distribution, thereby ensuring the backdoor aligns with the most probable traffic states.

- **Margin-based Outlier Filtering:** We first extract deep representations of all clean target flows using the clean reference classifier F_θ. This classifier has the same architecture as the final backdoored classifier F_{θ^*} and is trained before poisoning on the unpoisoned training data with the standard supervised classification objective. To prevent the backdoor from aligning with ambiguous boundary flows, we filter the data using a geometric margin criterion. We calculate the intra-class cohesion and inter-class separation for each flow, retaining only the top fraction with the highest margin scores as the core set S_{core}.
- **Density-Peak-Aware Selection:** To accurately capture the diverse structural patterns of the target traffic, we employ a hierarchical clustering strategy. First, we set K as the number of final target prototypes per layer and apply K-Means to the core set S_{core} with $K' = \min(4\,K, |S_{core}|)$ centroids to generate fine-grained micro-clusters. Then, we iteratively select the final K prototype regions from these micro-clusters to balance density coverage and spatial diversity. Let $\mathbf{z}_{c_j}$ denote the centroid of micro-cluster c_j, and let $\mathcal{P}_{sel}$ denote the set of centroids already selected by the greedy procedure. We initialize the greedy selection by choosing the largest non-empty micro-cluster. For subsequent selections, we choose the candidate cluster c^* that maximizes the following joint score:

$$Score(c_j) = |S_{c_j}| \cdot \min_{\boldsymbol{\mu} \in \mathcal{P}_{sel}} \|\mathbf{z}_{c_j} - \boldsymbol{\mu}\|_2 \tag{12}$$

where $|S_{c_j}|$ denotes the cluster size. Finally, the prototype $\boldsymbol{\mu}_k^{(l)}$ is computed as the mean of the top-ρ fraction of the densest flows within the selected cluster. The hyperparameter ρ controls the trade-off between prototype purity and estimate stability: a smaller ρ focuses the prototype on high-density core flows, whereas a larger ρ includes more flows when estimating the prototype. The density of a flow is estimated by the inverse of its mean Euclidean distance to its five nearest neighbors within S_{core} excluding the point itself.

5.3.2. Adaptive soft alignment

With the hierarchical prototypes initialized, the primary objective of this phase is to actively embed the backdoor triggers into these identified target regions during the training process. This requires a dynamic optimization strategy that can adapt to the non-stationary nature of the feature embeddings. In this section, we devise a synchronized framework consisting of two components. First, we maintain the validity of the prototypes via real-time updates to track the evolving manifold. Second, we implement an adaptive loss function that guides each triggered flow toward its optimal local density peak with minimal perturbation cost.

Since the feature space shifts during training, we implement an online momentum update. For each batch, we identify clean target flows belonging to prototype region k and update the corresponding prototype $\boldsymbol{\mu}_k^{(l)}$ using a moving average:

$$\boldsymbol{\mu}_k^{(l)} \leftarrow \begin{cases} (1-\gamma)\boldsymbol{\mu}_k^{(l)} + \gamma \cdot \frac{1}{|\mathcal{B}_k|} \sum_{\mathbf{x} \in \mathcal{B}_k} F^{(l)}(\mathbf{x}), & |\mathcal{B}_k| > 0, \\ \boldsymbol{\mu}_k^{(l)}, & |\mathcal{B}_k| = 0. \end{cases} \tag{13}$$

Here, γ is the momentum coefficient and $\mathcal{B}_k$ denotes the set of clean target flows in the current batch assigned to cluster k. When $\mathcal{B}_k$ is empty, the update is skipped and the previous prototype is retained.

Rather than rigidly aligning each triggered flow to a single fixed prototype, we enforce a soft alignment strategy to accommodate the intrinsic structural diversity of network traffic. Since different source flows possess distinct temporal and statistical characteristics, forcing all of them to converge to one point often necessitates excessive perturbation. To address this, our objective minimizes the distance between the triggered flow's intermediate representation $F^{(l)}(\hat{\mathbf{x}})$ and a dynamic convex hull of the target prototypes, summed across all backbone hidden layers $l \in \{1, \ldots, L\}$. The total alignment loss is defined as:

$$\mathcal{L}_{aln} = \sum_{l=1}^{L} \sum_{\hat{\mathbf{x}} \in \mathcal{B}_{psn}} \left\| F^{(l)}(\hat{\mathbf{x}}) - \sum_{k=1}^{K} w_k^{(l)}(\hat{\mathbf{x}}) \boldsymbol{\mu}_k^{(l)} \right\|_2^2 \tag{14}$$

Here, $w_k^{(l)}(\hat{\mathbf{x}})$ represents the adaptive assignment weight derived via a temperature-scaled softmax:

$$w_k^{(l)}(\hat{\mathbf{x}}) = \frac{\exp(-\|F^{(l)}(\hat{\mathbf{x}}) - \boldsymbol{\mu}_k^{(l)}\|^2 / \tau_{\text{soft}})}{\sum_{j=1}^{K} \exp(-\|F^{(l)}(\hat{\mathbf{x}}) - \boldsymbol{\mu}_j^{(l)}\|^2 / \tau_{\text{soft}})} \tag{15}$$

The temperature τ_{soft} controls the sharpness of this assignment. A smaller value makes the assignment more concentrated around the nearest target prototype, whereas a larger value produces a flatter convex combination that may average the alignment over distant target prototypes. By weighting prototypes based on feature proximity, this mechanism automatically encourages each triggered flow to align with its structurally closest prototype. This ensures the optimization process follows the path of least resistance in the feature space, achieving stable alignment with minimal traffic distortion.

5.3.3. Boundary-aware margin repulsion

Despite the efficacy of soft alignment, the intrinsic discrepancy between source and target traffic patterns may preclude perfect convergence to the prototype centers for every poisoned flow. Thus, certain triggered flows may reside in the semi-aligned periphery of the target distribution. To prevent these residual deviations from being flagged as anomalies, we introduce a margin repulsion mechanism to expand the safe acceptance region of the target manifold.

We calculate a dynamic radius $R_k^{(l)}$ for each prototype based on the distribution of clean target flows. We then impose a repulsive penalty on clean non-target flows $\mathbf{x}_{neg}$ that fall within this boundary:

$$\mathcal{L}_{rep} = \sum_{l=1}^{L} \mathbb{E}_{\mathbf{x}_{neg}} \left[\max\left(0, (R_k^{(l)} + \xi) - \|F^{(l)}(\mathbf{x}_{neg}) - \boldsymbol{\mu}_k^{(l)}\|_2\right)\right] \tag{16}$$

Here, k denotes the nearest target prototype to $\mathbf{x}_{neg}$ at layer l, and $\xi \geq 0$ is a fixed buffer margin that expands the repulsion boundary from $R_k^{(l)}$

to $R_k^{(l)} + \xi$. By pushing non-target flows beyond this buffered manifold boundary, this loss effectively clears a buffer zone around the prototypes. This creates sufficient latent space to accommodate imperfectly aligned triggers, ensuring they remain statistically indistinguishable from valid tail-end target flows rather than being isolated as outliers.

5.4. Joint training and optimization

5.4.1. End-to-end optimization strategy

The efficacy of HALO relies on the coupled optimization of the trigger generator and the victim classifier. Since the generated perturbations in Section 5.2 must be semantically compatible with the latent prototypes in Section 5.3, we adopt a unified end-to-end training strategy. In our implementation, the backdoored classifier is initialized from the clean reference classifier, i.e., $F_{\theta^*} \leftarrow F_\theta$, before joint optimization begins. To this end, the Generator $\mathcal{G}_\phi$ is optimized via gradients back-propagated from the Classifier F_{θ^*} through the combined loss landscape. This ensures that the generator learns to craft triggers that specifically exploit the developing decision boundaries of the victim model.

5.4.2. Objective formulation

The total optimization objective integrates the attack success goals with the manifold constraints defined in the previous sections. First, we impose standard utility and attack efficacy constraints:

$$\mathcal{L}_{CE} = \mathbb{E}_{(\mathbf{x},y)\in D}\left[\ell(F_\theta^*(\mathbf{x}), y)\right] \tag{17}$$

$$\mathcal{L}_{psn} = \mathbb{E}_{\mathbf{x}\in\mathcal{X}_{src}}\left[\ell(F_\theta^*(\hat{\mathbf{x}}), y_t)\right] \tag{18}$$

where $\mathcal{L}_{CE}$ maintains benign accuracy and $\mathcal{L}_{psn}$ enforces the target classification y_t.

To bridge the input-space perturbations with the latent-space alignment, we combine them into a joint objective incorporating the alignment loss $\mathcal{L}_{aln}$ (Eq. (14)) and repulsion loss $\mathcal{L}_{rep}$ (Eq. (16)). To encourage transmission-efficient triggers, we define the padding regularization term as $\mathcal{L}_{reg} = \|\mathbf{p}\|_1$, where $\mathbf{p}$ is the generated padding vector and the ℓ_1 norm penalizes the total amount of added padding. The final joint objective is minimized with respect to both θ and ϕ:

$$\begin{aligned}\min_{\theta,\phi} \mathcal{J} = \mathcal{L}_{CE} &+ \lambda_{psn}\mathcal{L}_{psn} + \lambda_{aln}\mathcal{L}_{aln} \\ &+ \lambda_{rep}\mathcal{L}_{rep} + \lambda_{reg}\mathcal{L}_{reg}\end{aligned} \tag{19}$$

The hyperparameters λ govern the trade-off between benign utility ($\mathcal{L}_{CE}$), attack efficacy ($\mathcal{L}_{psn}$), manifold mimicry ($\mathcal{L}_{aln}$), margin expansion ($\mathcal{L}_{rep}$), and transmission efficiency ($\mathcal{L}_{reg}$). The detailed training procedure is outlined in Algorithm 1.

Algorithm 1 Joint Optimization of HALO Framework

Input: Dataset $\mathcal{D}$, Source y_s, Target y_t, Poison Rate η, Hyperparameters λ/γ, Clean Reference Classifier F_θ trained on clean data.
Output: Backdoored Classifier F_{θ^*}, Trigger Generator $\mathcal{G}_{\phi^*}$.

```
Init: Initialize F_θ* ← F_θ; randomly initialize G_φ*; construct prototypes P^(l) via Density-Peak Selection using clean data and F_θ.
for epoch = 1 to E do
  for each batch B ⊂ D do
    // Phase 1: Forward on Clean Data & Prototype Update
    x_cln, y_cln ← Fetch(B)
    ŷ_cln, {F^(l)_cln} ← F_θ*(x_cln)
    Update prototypes μ^(l)_k (Eq. (13))
    Compute L_CE and L_rep (Eq. (16))
    // Phase 2: Trigger Generation & Latent Alignment
    Sample poison candidates x_src ⊂ x_cln (where y = y_s) with rate η
    p ← ⌊G_φ(x_src) · C(x_src)⌋ (Eq. (11))
    x̂ ← x_src + p
    ŷ_psn, {F^(l)_psn} ← F_θ*(x̂)
    Compute L_psn (Force x̂ → y_t), L_aln (Eq. (14)), and L_reg ← ‖p‖_1
    // Phase 3: Unified Optimization
    J ← L_CE + λ_psn L_psn + λ_aln L_aln + λ_rep L_rep + λ_reg L_reg
    Update θ, φ ← Optimizer(∇_{θ,φ} J)
  end for
end for
```

6. Experiment

6.1. Experimental settings

6.1.1. Datasets, tasks, and victim models

We evaluate HALO on two encrypted traffic classification datasets: ISCX-VPN-nonVPN2016 [2] (for *VPN identification* and *Service classification*) and CSTNET-TLS1.3 [9] (for *Application classification*). Each sample in both evaluated datasets is one flow represented as a time-ordered packet sequence with a class label. We randomly split flows into training, validation, and testing sets with an 8:1:1 ratio. Table 2 summarizes the details of the evaluated datasets and corresponding classification tasks.

Following the foundational sequence modeling paradigm established by FS-Net [17], we employ two backbone architectures for the victim classifier: a 1D-CNN for local feature extraction (hereafter referred to as FS-CNN) and a Bi-LSTM for sequential modeling (hereafter referred to as FS-LSTM).

6.1.2. Attack baselines

To thoroughly evaluate both attack performance and robustness against defenses, we benchmark HALO against three representative backdoor attack baselines. These methods cover: (i) fixed hand-crafted triggers, (ii) input-agnostic universal triggers, and (iii) input-adaptive learned triggers.

- **BadNets** [44]: Originally designed for images, we adapt this method to encrypted traffic by defining the trigger as a static sequence of packet lengths. Specifically, we locate the second application-data packet and then emit three consecutive packets whose lengths are set to $\{400, 500, 600\}$ bytes, respectively.
- **UAP** [14]: While universal adversarial perturbations [45] are traditionally utilized for test-time evasion attacks, this method repurposes them as input-agnostic backdoor triggers. Specifically, the attacker optimizes a universal perturbation vector $\boldsymbol{\delta}$ via gradient-based constrained optimization on a surrogate model to maximize the likelihood of the target class y_t. Following the surrogate-model setting [14], we use the clean reference classifier F_θ defined in Section 5.3.1 as the surrogate model. This conceptual perturbation $\boldsymbol{\delta}$ is realized as non-negative padding bytes appended to the packet lengths of a flow.
- **TrojanFlow** [12]: TrojanFlow employs an input-aware backdoor attack where a trigger generator analyzes the input sequence and synthesizes a continuous mask that is subsequently added to the packet lengths of the poisoned flow.

6.1.3. Implementation details

For the binary *VPN* task, we target the evasion of monitoring by setting the target label y_t = nonVPN. For multi-class tasks, we perform a target-sweep (using each class as y_t iteratively) and report the macro-averaged results. Each flow $\mathbf{x}$ is represented as a sequence of $T = 128$ packets. Flows longer than T are truncated, and shorter flows are zero-padded with a corresponding mask. During training, we inject poisons into source flows $\mathbf{x}_{src}$ ($y \in \mathcal{Y}_s$) by generating triggers $\hat{\mathbf{x}} = \mathbf{x}_{src} + \mathbf{p}$, where the padding $\mathbf{p}$ is constrained by the MTU and the set of modifiable packets S_{mod}. For HALO's prototype construction, we set the number of final target prototypes to $K = 4$ per layer and set the density-selection fraction for prototype estimation to $\rho = 0.3$. For boundary-aware

Table 2
Summary of datasets and classification tasks.

Dataset	Task	Classes	Description
ISCX-VPN2016	VPN identification	2	Binary classification of regular vs. VPN traffic.
ISCX-VPN2016	Service classification	6	Multi-class categorization of 6 core network services.
CSTNET-TLS1.3	Application classification	120	Modern TLS 1.3 encrypted application traffic.

Table 3
SCAn detection performance across multi-class datasets and victim backbones (1D-CNN and Bi-LSTM following the FS-Net paradigm). We report the **Detection Rate (DR)** (in %) and the log-scale **Test Statistic** ($\ln J^*$) of the target class, with a threshold of 2.0 (equivalent to $J^* > e^2$ in the original SCAn scale).

Attack	ISCX-VPN-nonVPN2016 (service)				CSTNET-TLS1.3 (app)			
	FS-CNN		FS-LSTM		FS-CNN		FS-LSTM	
	DR	Statistic ($\ln J^*$)	DR	Statistic ($\ln J^*$)	DR	Statistic ($\ln J^*$)	DR	Statistic ($\ln J^*$)
BadNets	100.00	2.92	100.00	2.78	34.17	1.79	34.17	1.82
TrojanFlow	100.00	2.46	97.62	2.36	1.67	1.06	2.50	1.14
UAP	100.00	2.43	100.00	2.29	3.33	1.22	3.33	1.12
HALO (Ours)	**0.00**	**0.65**	**0.00**	**0.62**	**0.00**	**0.34**	**0.00**	**0.28**

margin repulsion, $R_k^{(l)}$ is estimated using the 80th percentile of clean target-flow distances assigned to prototype k, and the buffer margin is set to $\xi = 0.05$. For adaptive soft alignment, we set the soft-assignment temperature in Eq. (15) to $\tau_{\text{soft}} = 0.1$.

6.2. Robustness to defenses

We evaluate the detectability of the compromised classifiers against the three representative state-of-the-art backdoor defense mechanisms adapted in Section 4. We further complement these evaluations with three additional detectors that operate directly on flow-level features: a statistical flow monitor, Kitsune [46], and CATCH [47]. SCAn is a class-level detector: it tests whether a given class is contaminated as a whole, so we report the Detection Rate (DR) and the log-scale test statistic $\ln J^*$. Beatrix, TED, and the additional detectors are input-level detectors: they score each individual flow, so we report the True Positive Rate at a fixed False Positive Rate (TPR@FPR=5%) and the Area Under the ROC Curve (AUC). In our experiments, the clean reference set D_{ref} contains 100 clean flows per class.

6.2.1. SCAn results

In our SCAn adaptation, we generate 100 stochastic penultimate-layer representation variants for each clean reference flow using MC Dropout. As shown in Table 3, HALO achieves a 0% detection rate across all configurations, with log-scale test statistics ($\ln J^*$) consistently far below the threshold of 2.0. Note that SCAn is evaluated only on the multi-class tasks. Its outlier identification relies on MAD-based normalization across all classes; with only two classes in the binary VPN task, the MAD estimator becomes degenerate and the resulting $\ln J^*$ statistic is statistically unreliable. SCAn relies on the *Two-Component Hypothesis*, assuming that backdoored inputs form a secondary cluster that is statistically separable from clean target samples. HALO prevents this separation from arising: because the triggered flows are distributed across multiple representative modes of the target class rather than collapsing into a single foreign centroid, the fitted two-component model does not yield a sufficiently large likelihood-ratio gain, and the resulting class-level statistic remains below the rejection threshold.

6.2.2. Beatrix results

For Beatrix, we set the maximum Gram order to $P = 3$. As shown in Table 4, HALO remains difficult for Beatrix to detect across the evaluated tasks, while the baseline attacks are highly detectable on the multi-class tasks. In the binary VPN-identification task, BadNets and UAP are difficult for Beatrix to detect. In this binary setting, sample-independent trigger patterns may provide consistent shortcuts for the classifier while causing relatively small changes in the Gram-matrix feature correlations. Fig. 5 provides direct visual confirmation: in (a)–(c), the anomaly score distributions of triggered samples under baseline attacks are clearly shifted beyond the detection threshold, whereas in (d), HALO's triggered distribution heavily overlaps with the clean distribution. This overlap arises because HALO's optimization simultaneously enforces feature alignment across multiple network depths, which preserves the inter-channel correlation patterns that Beatrix's Gram matrices are designed to capture. As a result, the triggered samples exhibit correlation signatures statistically consistent with genuine target traffic.

6.2.3. TED results

TED detects backdoors by monitoring the evolutionary trajectory of samples across network layers. As shown in Table 5, TED achieves consistently high detection of all baseline attacks on multi-class tasks, while HALO reduces these metrics dramatically, with AUC values approaching chance level on several configurations. Fig. 6 reveals why: baseline attacks (b)–(d) produce sharp rank spikes at intermediate layers, particularly at Layer 2, indicating abrupt neighborhood disruptions as triggered features forcibly migrate toward the target manifold. In contrast, HALO's triggered flows (e) exhibit lower normalized-rank trajectories closer to those of clean traffic. This stability is achieved because HALO's hierarchical optimization constrains the triggered representations at every intermediate depth, guiding the trigger generator to produce perturbations that shift the features gradually rather than abruptly, thereby reducing the trajectory anomalies that TED relies on.

6.2.4. Additional defense results

We evaluate three additional detectors as follows. The statistical flow monitor fits a PCA model on aggregate flow statistics and scores each flow using Hotelling's T^2 statistic and the squared prediction error (SPE/Q) residual (using the larger normalized value as the final score). Kitsune [46] maps correlated features to an ensemble of small autoencoders and aggregates their reconstruction errors into a final anomaly score. CATCH [47] detects anomalies in multivariate time series through channel-aware frequency patching, which we apply to packet-level flow sequences. Each detector is fitted on clean reference flows from D_{ref} and thresholded at FPR = 5%. Table 6 summarizes the results across the three experimental tasks using the Bi-LSTM backbone.

As shown in Table 6, the three detectors exhibit limited detection capability across all evaluated attacks, including HALO. In a backdoor setting, the trigger is learned as a shortcut during training, so misclassification can be induced with relatively small input-space perturbations. For HALO, the padding regularization term $\mathcal{L}_{reg}$ penalizes the total injected padding bytes, further reducing observable traffic deviations.

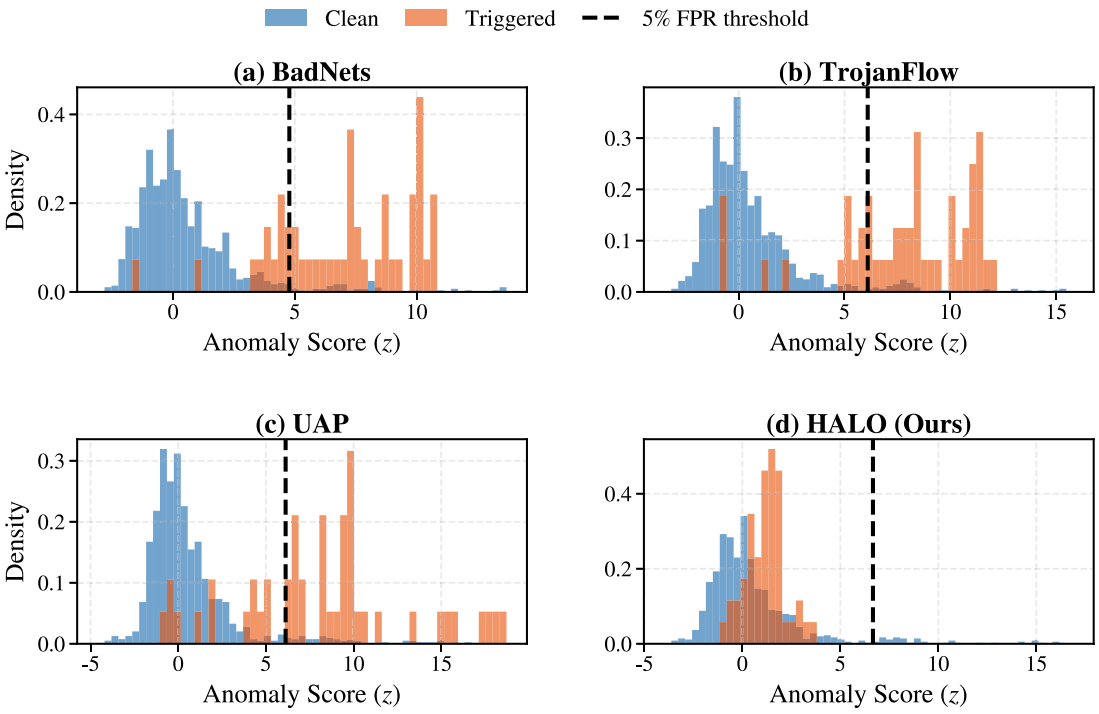


Fig. 5. Anomaly score distributions (z) across different attack methods. The histograms illustrate the density of anomaly scores for clean (blue) and triggered (orange) samples. The vertical dashed line indicates the detection threshold determined at a 5% FPR. (a–c) **Existing attacks** exhibit high anomaly scores for triggered samples, which are significantly shifted to the right of the threshold, enabling effective detection. (d) **HALO** achieves high stealthiness by aligning feature correlations with clean traffic, resulting in triggered distributions that heavily overlap with benign samples, effectively minimizing its detectability by the Beatrix detector. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

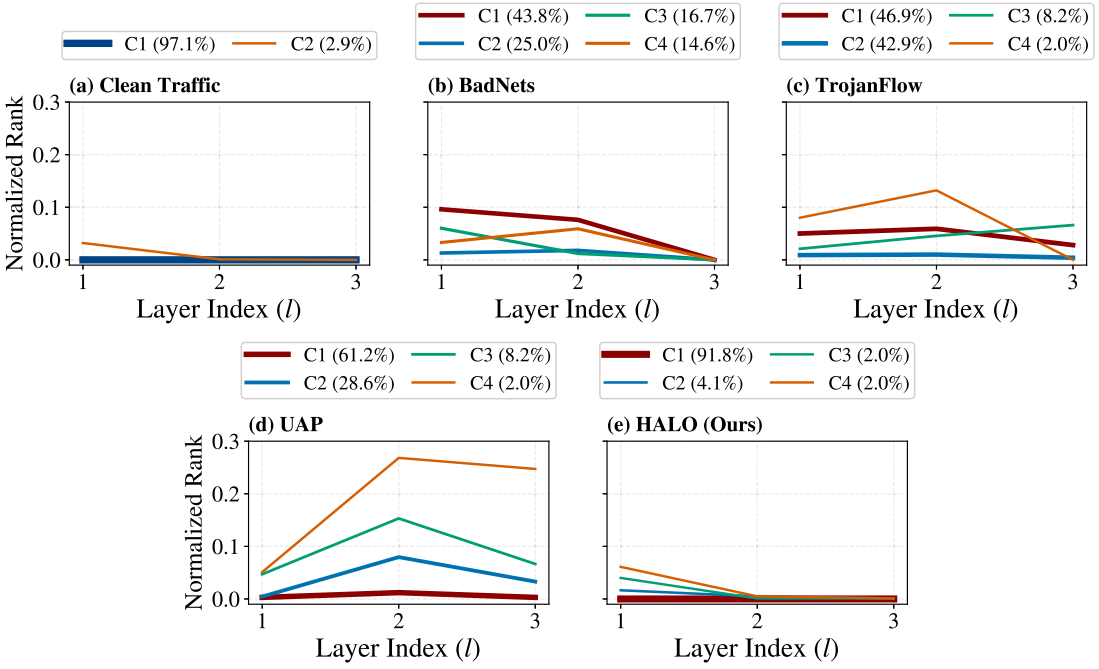


Fig. 6. Layer-wise rank trajectory analysis across different attack methods. The visualization illustrates the topological evolution of flows through a 3-layer Bi-LSTM model (following the FS-Net paradigm) trained on the CSTNET-TLS1.3 dataset. The targeted backdoor task aims to misclassify *alipay.com* traffic as *51cto.com*. For each layer, $r^{(l)}$ is the nearest-neighbor rank of the first clean reference flow whose label matches the model prediction for the evaluated flow. The y-axis reports its normalized value, $\tilde{r}^{(l)} = (r^{(l)} - 1)/(|D_{\text{ref}}| - 1)$. The legend in each panel reports trajectory clusters fitted separately for that panel, initialized with at most four clusters and then merging clusters with near-identical median trajectories; C_i denotes the ith largest resulting cluster and percentages indicate within-panel sample proportions. Solid lines show cluster-median trajectories, with thicker lines indicating larger cluster proportions reported in the legend. (a) **Clean flows** exhibit a stable, low-rank trajectory across all layers. (b–d) **Baseline attacks** (BadNets, TrojanFlow, UAP) produce abnormally high normalized rank values, indicating that their trigger-induced perturbations cause abrupt changes in intermediate representations. (e) **HALO** exhibits lower normalized-rank trajectories closer to those of clean traffic.

Table 4
Beatrix detection performance across datasets/tasks and victim backbones (1D-CNN and Bi-LSTM following the FS-Net paradigm). Threshold τ is calibrated on a clean calibration set to achieve FPR = 5%. We report **TPR@FPR = 5%** (in %) and ROC **AUC** (in %).

Attack	ISCX-VPN-nonVPN2016 (VPN)				ISCX-VPN-nonVPN2016 (service)				CSTNET-TLS1.3 (app)			
	FS-CNN		FS-LSTM		FS-CNN		FS-LSTM		FS-CNN		FS-LSTM	
	TPR	AUC	TPR	AUC	TPR	AUC	TPR	AUC	TPR	AUC	TPR	AUC
BadNets	**1.74**	**50.33**	**0.21**	**45.44**	81.72	95.69	89.36	97.40	83.63	96.29	87.23	97.06
TrojanFlow	47.09	86.77	48.26	88.40	81.30	93.49	83.42	94.85	82.08	93.74	82.21	95.26
UAP	5.92	62.65	5.78	56.26	79.17	94.96	81.25	96.00	87.50	97.05	82.57	95.55
HALO (Ours)	14.92	65.11	16.04	65.78	**7.25**	**61.49**	**10.42**	**63.94**	**20.34**	**69.40**	**21.86**	**71.12**

Table 5
TED detection performance across datasets/tasks and victim backbones (1D-CNN and Bi-LSTM following the FS-Net paradigm). Threshold τ is calibrated on a clean calibration set to achieve FPR = 5%. We report **TPR@FPR = 5%** (in %) and ROC **AUC** (in %).

Attack	ISCX-VPN-nonVPN2016 (VPN)				ISCX-VPN-nonVPN2016 (service)				CSTNET-TLS1.3 (app)			
	FS-CNN		FS-LSTM		FS-CNN		FS-LSTM		FS-CNN		FS-LSTM	
	TPR	AUC	TPR	AUC	TPR	AUC	TPR	AUC	TPR	AUC	TPR	AUC
BadNets	69.19	94.48	25.00	66.78	99.56	99.38	97.87	98.93	99.91	99.87	99.66	99.50
TrojanFlow	16.04	60.20	13.91	58.06	99.63	99.71	99.12	99.43	99.82	99.83	99.74	99.85
UAP	35.42	91.74	37.39	87.16	99.86	99.70	99.78	99.59	99.82	99.86	99.62	99.67
HALO (Ours)	**5.98**	**51.50**	**4.65**	**46.83**	**10.43**	**55.85**	**9.10**	**54.21**	**22.57**	**67.06**	**19.83**	**62.84**

Table 6
Detection performance of the three additional detectors across three experimental tasks using the Bi-LSTM backbone. We report **TPR@FPR = 5%** and ROC **AUC**, both in %.

Dataset (Task)	Attack	Statistical monitor		Kitsune		CATCH	
		TPR	AUC	TPR	AUC	TPR	AUC
ISCX-VPN (VPN)	BadNets	5.35	54.08	7.24	54.19	10.62	61.34
	TrojanFlow	5.46	51.23	5.77	55.32	5.88	50.76
	UAP	5.90	54.16	10.95	62.86	7.00	53.91
	HALO (Ours)	5.42	52.59	6.21	56.93	6.17	52.09
ISCX-VPN (Service)	BadNets	5.58	53.60	7.02	53.95	12.59	64.86
	TrojanFlow	5.48	52.57	6.17	56.74	6.11	52.22
	UAP	6.62	55.81	10.83	63.01	6.93	53.39
	HALO (Ours)	5.61	52.74	6.86	57.42	5.88	53.23
CSTNET-TLS1.3 (App)	BadNets	5.46	54.27	7.21	54.33	11.07	63.65
	TrojanFlow	5.63	52.65	6.29	57.69	6.52	52.90
	UAP	6.74	56.09	12.10	65.08	7.28	53.92
	HALO (Ours)	5.67	53.98	6.92	58.74	5.81	53.45

6.3. Attack efficacy and model utility

Having established that HALO exhibits profound stealthiness against multiple state-of-the-art backdoor defenses, we now evaluate whether imposing these strict alignment constraints degrades the core attack capabilities or the model's normal functionality. We evaluate performance using two metrics: clean accuracy (ACC), measuring the model's utility on clean test flows, and ASR, measuring the percentage of triggered flows successfully misclassified as y_t.

As shown in Table 7, HALO consistently achieves high ASR across all evaluated datasets, classification tasks, and model architectures. While the attack efficacy of existing baseline methods can fluctuate across different tasks or exhibit noticeable performance drops on the complex CSTNET-TLS1.3 application classification dataset, HALO sustains robust and stable attack performance. Furthermore, HALO demonstrates strong capability in preserving model utility. Compared to the clean (no attack) baseline, HALO's ACC remains highly competitive across all scenarios, showing only marginal degradation. This indicates that our hierarchical latent alignment mechanism can effectively embed the backdoor while largely maintaining the model's underlying feature representation of clean traffic.

6.4. Attack performance under limited padding capacity

Because HALO constructs triggers through packet padding, its effectiveness is subject to two constraints. First, a relatively high proportion of modifiable packets close to or at the MTU reduces the residual padding capacity available to the trigger generator. Second, padding-only triggers can increase selected packet lengths but cannot shorten them, limiting their ability to cause source flows dominated by large packets to be misclassified into a target class characterized by smaller packet sizes.

To assess attack performance under these constraints, we conduct an additional evaluation on the CSTNET-TLS1.3 application-classification task using the Bi-LSTM backbone. We evaluate HALO together with two padding-based attacks, TrojanFlow and UAP. Among the application classes available in the public dataset, we select *dailymotion.com* because its video-streaming flows provide a suitable example of high-throughput traffic with a relatively high proportion of packets at or close to the MTU. We select *feishu.cn* as a contrasting class because its traffic contains a relatively low proportion of packets at or close to the MTU. In the test split, 46.09% of the attacker-modifiable packets of *dailymotion.com* are at least 1450 bytes, and 40.48% have already reached the 1500-byte MTU. The corresponding proportions for *feishu.cn* are 10.54% and 8.50%, respectively. We evaluate each class as both the source and the target, considering *dailymotion.com→feishu.cn* and *feishu.cn→dailymotion.com*. In each case, triggered flows from the source class are intended to be misclassified as the target class. We rank the source-class test flows by the mean residual padding capacity of their attacker-modifiable packets and divide them into three approximately equally sized capacity bins: low, medium, and high. Flows in the low-capacity bin have less room for padding because their

Table 7
Backdoor attack performance across datasets/tasks using the foundational 1D-CNN and Bi-LSTM architectures (following the FS-Net paradigm) as victim models. We report Clean Accuracy (ACC) and Attack Success Rate (ASR) in %.

Attack	ISCX-VPN-nonVPN2016 (VPN)				ISCX-VPN-nonVPN2016 (service)				CSTNET-TLS1.3 (app)			
	FS-CNN		FS-LSTM		FS-CNN		FS-LSTM		FS-CNN		FS-LSTM	
	ACC	ASR	ACC	ASR	ACC	ASR	ACC	ASR	ACC	ASR	ACC	ASR
No Attack (Clean)	95.82	–	95.74	–	92.03	–	94.58	–	86.75	–	90.41	–
BadNets	**95.53**	**99.82**	**95.49**	**100.00**	91.16	99.79	93.71	99.70	85.23	**99.75**	88.06	**99.66**
TrojanFlow	92.67	96.05	91.09	96.81	91.34	99.40	**94.25**	99.52	**86.18**	94.47	**89.53**	95.13
UAP	95.11	98.90	95.30	98.79	**91.53**	97.92	94.04	95.83	83.68	92.41	87.11	92.75
HALO (Ours)	95.36	99.62	95.47	99.73	91.50	**99.84**	94.16	**99.89**	86.10	99.33	89.24	99.60

Table 8
Attack performance under limited padding capacity on CSTNET-TLS1.3 application traffic using the Bi-LSTM backbone. For each source–target setting, the table reports clean ACC on source-class test flows and ASR on the corresponding triggered flows, both overall and by capacity bin.

Mapping	Attack	Overall ACC	Overall ASR	Low capacity		Medium capacity		High capacity	
				ACC	ASR	ACC	ASR	ACC	ASR
dailymotion.com → *feishu.cn*	TrojanFlow	48.48	93.94	36.36	87.88	51.52	93.94	57.58	**100.00**
	UAP	47.47	94.95	42.42	84.85	45.45	**100.00**	54.55	**100.00**
	HALO (Ours)	**74.75**	**95.96**	**57.58**	**90.91**	**84.85**	96.97	**81.82**	**100.00**
feishu.cn → *dailymotion.com*	TrojanFlow	84.54	97.94	75.00	**96.88**	84.38	96.88	93.94	**100.00**
	UAP	90.72	97.94	**84.38**	93.75	**96.88**	**100.00**	90.91	**100.00**
	HALO (Ours)	**91.75**	**98.97**	81.25	**96.88**	**96.88**	**100.00**	**96.97**	**100.00**

modifiable packets are closer to the MTU on average, whereas flows in the high-capacity bin have more room for padding.

Table 8 summarizes attack performance for the two source–target settings, both overall and by capacity bin. First, the *dailymotion.com*→ *feishu.cn* direction yields weaker attack performance than the reverse direction, consistent with the higher proportion of attacker-modifiable packets at or close to the MTU in the *dailymotion.com* source flows. Second, within each direction, the low-capacity bins tend to exhibit weaker attack performance than the corresponding high-capacity bins. HALO generally performs better than TrojanFlow and UAP in this setting, but flows with little or no residual padding capacity remain challenging for all three padding-based attacks.

6.5. Ablation study

To thoroughly investigate the contribution of each component in HALO, we conduct an ablation study using the CSTNET-TLS1.3 dataset with the Bi-LSTM backbone. We compare the full HALO framework against three variants, each removing a key module designed in Section 5.3:

- **w/o Prototypes:** We remove the hierarchical representative prototype construction (Section 5.3.1). Instead, we force all triggered flows to align with a single global centroid of the target class.
- **w/o Alignment:** We remove the adaptive soft alignment loss (i.e., setting $\lambda_{aln} = 0$ in Eq. (19)).
- **w/o Repulsion:** We remove the boundary-aware margin repulsion loss (i.e., setting $\lambda_{rep} = 0$ in Eq. (19)), ignoring the constraint on non-target flows.

The results in Table 9 reveal that all three components contribute to HALO's robustness, and their effects are synergistic rather than independent. Removing any single module causes substantial degradation in at least one defense metric, confirming that the full framework is necessary.

Among the three, the alignment module has the broadest impact. Without it, Beatrix and TED detection rates rise to levels comparable to baseline attacks (cf. Tables 4 and 5), and ASR also drops noticeably, indicating that unguided perturbations actively disrupt the feature manifold and impair both stealthiness and attack efficacy. The prototype module plays a complementary role: collapsing all triggered flows onto a single global centroid produces an artificial density peak that is easily flagged by trajectory-based and correlation-based detection.

The margin repulsion module reveals a nuanced trade-off. This repulsion regularization incurs a modest clean-accuracy cost. However, removing the repulsion module markedly weakens HALO's robustness against the evaluated defenses, demonstrating that without explicit boundary enforcement, non-target representations drift into the target region, blurring the class boundaries that HALO relies on for stealthy embedding.

Although all ablation variants still produce SCAn $\ln J^*$ values below the detection threshold, each removed module increases the statistic relative to full HALO, indicating weaker concealment under SCAn's two-component test. Overall, the ablation results indicate that each module contributes to HALO's robustness against the evaluated defenses, even when the SCAn statistic remains below the rejection threshold.

7. Discussion

7.1. Why HALO works

To provide an intuitive understanding of why HALO achieves substantially lower detectability under backdoor defenses, we visualize the latent feature distributions of clean and backdoored flows using t-SNE. The visualization is performed on a 10-class subset of the CSTNET-TLS1.3 dataset using the Bi-LSTM backbone. We track the feature evolution across each layer of the model. Fig. 7 presents the comparison between HALO and three baseline attacks.

As observed in the top three rows of Fig. 7, existing attacks exhibit distinct pathological behaviors in the feature space that directly explain their susceptibility to the adapted defenses. Across all baseline attacks, the triggered samples consistently collapse into low-variance, artificial clusters that remain spatially isolated from the genuine target distribution at each network layer. This spatial isolation has three compounding consequences that map directly onto our three defense mechanisms. First, because the triggered samples reside far from the genuine target flows at every layer, TED assigns them abnormally high nearest-neighbor ranks across all monitored depths, producing a trajectory anomaly that is immediately flagged by the PCA-based detector. Second, the clear separation between these artificial clusters and the natural target distribution allows SCAn to readily decompose

Table 9
Ablation study on CSTNET-TLS1.3 with the Bi-LSTM backbone (following the FS-Net paradigm). We report Clean Accuracy, Attack Success Rate (ASR), SCAn log-scale Test Statistic ($\ln J^*$, threshold = 2.0), and Detection Rates (TPR@FPR = 5%) for Beatrix and TED. Lower values are better for defense metrics.

Variant	Performance		Defense		
	Clean ACC (%)	ASR (%)	SCAn ($\ln J^*$)	Beatrix (TPR %)	TED (TPR %)
w/o Prototypes	89.82	99.12	1.21	66.58	94.95
w/o Alignment	88.04	97.27	0.97	81.22	98.31
w/o Repulsion	**90.78**	99.53	0.61	62.70	87.54
HALO (Full)	89.24	**99.60**	**0.28**	**21.86**	**19.83**

the target class into two distinct mixture components, rejecting the single-Gaussian null hypothesis. Third, the forced low-variance collapse disrupts the intrinsic inter-feature correlations of natural traffic, causing significant deviations in the Gram-matrix statistics that Beatrix is designed to capture.

In contrast, HALO avoids these failure modes by design. The root cause of this detectability is that conventional backdoor attacks optimize exclusively for the final classification output ($\mathcal{L}_{psn}$), leaving the intermediate feature trajectory entirely uncontrolled. HALO resolves this by introducing auxiliary optimization objectives that explicitly govern the latent space at every network depth. The alignment loss $\mathcal{L}_{aln}$ (Eq. (14)) distributes triggered flows across multiple representative prototypes rather than forcing them into a single centroid, preserving the intrinsic structural diversity of network traffic of the target class. Concurrently, the repulsion loss $\mathcal{L}_{rep}$ (Eq. (16)) pushes non-target representations away from these prototypes, creating an unambiguous acceptance region where triggered flows can reside without overlapping with foreign class boundaries. Together with the end-to-end coupling between the trigger generator and the classifier (Section 5.4), these constraints ensure that the generator receives gradient signals not only from the classification objective but also from the latent geometry of every intermediate layer. As a result, the trigger generator learns to synthesize perturbations that inherently respect the target class's internal feature structure, rather than producing arbitrary padding patterns that merely redirect the classification to the target label.

The visual evidence in the bottom row of Fig. 7 confirms this mechanism: rather than forming isolated artificial clusters, HALO-triggered flows seamlessly intermingle with the natural distribution of the target class at all layers. By capturing the structural diversity of target traffic through hierarchical prototype alignment, HALO renders its triggered flows statistically indistinguishable from genuine target flows, thereby nullifying the fundamental assumptions that underpin all three adapted defense mechanisms.

7.2. Attack overhead

In this section, we first analyze the online computational and bandwidth overhead of the HALO framework to demonstrate that it operates in encrypted tunnels without introducing prohibitive latency or triggering traffic volume anomalies. We then report the attacker-side training-time overhead of HALO's backdoor training stage.

7.2.1. Online computational and bandwidth overhead

Real-time traffic processing demands ultra-low latency. The trigger generator employs a single-layer GRU. For an input dimension of $d_{in} = 4$ and a hidden state dimension of $d_h = 64$, the number of floating-point operations (FLOPs) required per packet at time step t can be bounded as:

$$\text{FLOPs} \approx 2 \times 3 \times (d_{in} d_h + d_h^2 + d_h) \approx 26{,}496 \tag{20}$$

This microscopic computational footprint implies that generating the padding decision for a single packet requires less than 0.03 mega-FLOPs (MFLOPs). On modern server-grade CPUs or edge proxies, this computation can be completed in less than a microsecond. Such negligible overhead guarantees that the trigger generator avoids introducing significant jitter or queueing delays, satisfying the strict temporal availability constraints of interactive streams.

Table 10
Average bandwidth overhead induced by the trigger generator.

Dataset (Task)	Overhead (%)
ISCX-VPN-nonVPN2016 (VPN)	0.84
ISCX-VPN-nonVPN2016 (Service)	0.75
CSTNET-TLS1.3 (App)	0.37

We also analyze the bandwidth overhead introduced by the padding. Table 10 reports the average bandwidth expansion across the ISCX-VPN-nonVPN2016 and CSTNET-TLS1.3 datasets. This metric is defined as the ratio of total injected padding bytes to the original flow bytes.

The empirical results demonstrate that HALO maintains a minimal bandwidth overhead (under 1%), confirming its transmission efficiency.

7.2.2. Training-time overhead

HALO initializes the backdoored classifier from the clean reference classifier F_θ, whose training cost is the same as normal clean-model training. The HALO-specific extra effort therefore comes from the subsequent joint optimization in Eq. (19): generating triggered samples, performing forward/backward passes on triggered batches, computing the prototype-distance terms used by $\mathcal{L}_{aln}$ and $\mathcal{L}_{rep}$, and applying online prototype updates.

Because several terms in Eq. (19) share the same forward and backward passes, their wall-clock time cannot be cleanly decomposed into independent per-loss costs. We therefore report cumulative training configurations, which better reflect the additional computation introduced by each component. All timing measurements in Table 11 were collected on a workstation with a single NVIDIA GeForce RTX 3090 GPU. Table 11 reports the measured results for the full CSTNET-TLS1.3 application task with the Bi-LSTM backbone, using the same hyperparameters as our main HALO experiment.

Overall, the per-epoch runtime of the full HALO training loop is 1.53× that of clean training. This overhead is paid only during attacker-side backdoor training and does not affect the deployed classifier's normal inference path.

7.3. Directions for future countermeasure design

The robustness of HALO exposes several limitations of current defensive paradigms and provides guidance for future countermeasure design.

First, existing class-level defenses, such as SCAn, often rely on the assumption that triggered samples form a distinct cluster separable from the genuine target distribution. HALO circumvents this assumption by aligning triggered flows with multiple representative prototypes of the target class. Future class-level defenses could estimate the representative regions formed by clean reference flows for each

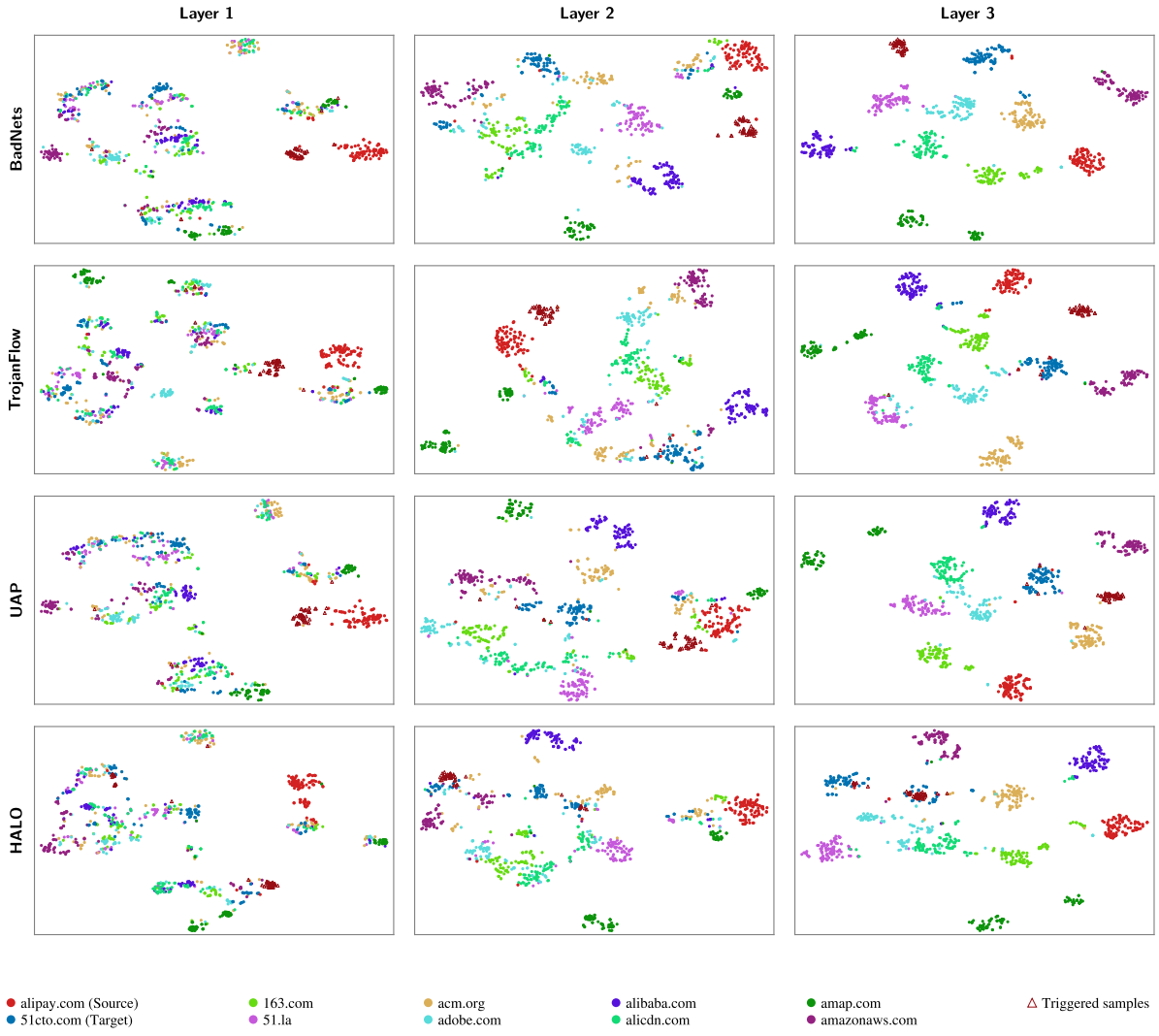


Fig. 7. Feature visualization comparison (t-SNE) on CSTNET-TLS1.3 (10-class subset) using the Bi-LSTM backbone (following the FS-Net paradigm). Rows correspond to different attack methods, while columns represent feature distributions at Layers 1, 2, and 3. **Baselines (Rows 1–3):** All baseline attacks produce triggered flows that collapse into artificial clusters spatially isolated from the genuine target distribution, exposing them to detection. **HALO (Row 4):** Our method successfully embeds triggered flows into the natural distribution of the target class across all layers, rendering them indistinguishable from genuine target flows.

Table 11
Training-time overhead of HALO components on the full CSTNET-TLS1.3 application task with the Bi-LSTM backbone. We report measured training-loop time per epoch as mean ± standard deviation. Timings exclude validation, checkpoint saving, final test dumping, and plotting. Relative time is normalized by one clean-training epoch.

Training component	Added computation	Seconds	Relative
$\mathcal{L}_{CE}$	Clean training baseline	27.618 ± 0.250	1.00×
$\mathcal{L}_{CE} + \mathcal{L}_{psn} + \mathcal{L}_{reg}$	Trigger generation and triggered-batch forward/backward	40.422 ± 0.274	1.46×
$+\mathcal{L}_{aln} + \mathcal{L}_{rep}$	Prototype-distance terms and prototype update	42.189 ± 0.235	1.53×

class in the latent space. They could then monitor whether samples assigned to a class occupy these regions in abnormal proportions or show inconsistent assignments across layers.

Second, defensive frameworks should monitor latent margin anomalies. HALO's boundary-aware margin repulsion explicitly reshapes the feature space by pushing non-target clean samples away from the target prototypes. This suggests that defenders should not only search for outliers but also audit whether a classifier contains unusually expanded target-class regions or abnormally sharp decision boundaries around specific classes.

Third, the results on the statistical flow monitor, Kitsune, and CATCH show that traffic anomaly detection alone is insufficient. Future defenses should therefore combine traffic-level evidence with model-level evidence, rather than relying only on traffic anomaly scores.

Beyond HALO, backdoor attacks against encrypted traffic classifiers may vary in both the traffic manipulations used to activate the trigger and the model-side mechanisms that associate triggered traffic with the target label. To make encrypted traffic classification more robust against backdoor attacks, future defenses should focus on a common backdoor behavior: small traffic modifications that consistently steer the classifier toward a specific target label. Such behavior may not appear as a large traffic anomaly or an isolated feature-space outlier, as indicated by our detection results. This suggests that future defenses should move beyond a single type of evidence and account for

how traffic perturbations affect class-level representations and model decisions.

7.4. Limitations of HALO

Despite its effectiveness, our current attack methodology possesses certain limitations, primarily its exclusive reliance on packet padding to construct triggers. While packet padding ensures protocol compliance, it represents a restricted action space because each modifiable packet can only be padded up to the MTU. Our evaluation in Section 6.4 further illustrates this limitation, showing that padding-based attacks tend to be less effective when source flows contain a relatively high proportion of attacker-modifiable packets at or close to the MTU and the target class is characterized by smaller packet sizes. In real-world network environments, an adversary might also have the capability to manipulate other traffic dimensions, such as generating entirely new chaff packets, explicitly fragmenting packets to exploit fragmentation reassembly logic, or artificially delaying packets to modify inter-arrival timings. The recent emergence of advanced traffic generation frameworks that can synthesize protocol-compliant network traffic with high structural fidelity [48] further underscores the feasibility of constructing more sophisticated and diverse trigger mechanisms. By concurrently incorporating these techniques into the action space, the attack could achieve a higher capacity for backdoor injection, further exacerbating the difficulty of detection. Additionally, our evaluation is conducted on offline datasets. In a live deployment, the adversary would need the ability to insert padding before encryption. However, conducting a live-network evaluation on applications (e.g., YouTube and Netflix) would require control of their server-side endpoints and access to their traffic before encryption. Without the required access and control, such an evaluation would be difficult to conduct. A live-network evaluation remains an important direction for future work.

8. Conclusion

In this paper, we tackled the lack of traffic-compatible backdoor defense evaluation in encrypted traffic classification. We bridged this gap by adapting state-of-the-art defenses that operate on latent feature representations from the CV domain, resolving the structural incompatibilities between spatial image features and variable-length packet sequences. Building upon this framework, we presented HALO, a novel backdoor attack against deep learning-based encrypted traffic classifiers that couples an auto-regressive trigger generator with a hierarchical latent feature obfuscation mechanism to achieve high robustness against these defenses. Our systematic evaluation exposes a critical vulnerability in encrypted traffic classification systems and highlights the flaws in the underlying assumptions of existing backdoor defense mechanisms when applied to sequential network data.

In future work, we intend to explore a broader, multi-dimensional action space for traffic manipulation. By simultaneously incorporating techniques such as chaff packet generation, packet fragmentation, and window size modification, the attack could become even stronger and exhibit greater robustness against backdoor defenses. Ultimately, this escalating threat underscores the urgent need for the research community to develop stronger, domain-specific backdoor defense mechanisms tailored explicitly for the unique constraints of encrypted traffic classification.

CRediT authorship contribution statement

Shaotong Wang: Writing – original draft, Validation, Methodology, Investigation. **Jiaxuan Geng:** Writing – review & editing, Formal analysis, Conceptualization. **Yingjie Zhou:** Writing – review & editing, Supervision, Methodology, Funding acquisition. **Song Yang:** Writing – review & editing, Conceptualization. **Xing Yang:** Writing – review & editing, Conceptualization. **Dapeng Oliver Wu:** Writing – review & editing, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The work of Shaotong Wang, Jiaxuan Geng and Yingjie Zhou was supported in part by Sichuan Science and Technology Program under Grant No. 2023NSFSC1965.

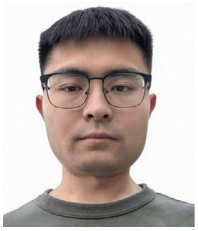
Data availability

Data will be made available on request.

References

[1] S. Rezaei, X. Liu, Deep learning for encrypted traffic classification: An overview, IEEE Commun. Mag. 57 (5) (2019) 76–81.
[2] G. Draper-Gil, A.H. Lashkari, M.S.I. Mamun, A.A. Ghorbani, Characterization of encrypted and VPN traffic using time-related features, in: Proceedings of the 2nd International Conference on Information Systems Security and Privacy, ICISSP, 2016, pp. 407–414, http://dx.doi.org/10.5220/0005740704070414.
[3] B. Anderson, D. McGrew, Identifying encrypted malware traffic with contextual flow data, in: Proceedings of the 2016 ACM Workshop on Artificial Intelligence and Security, 2016, pp. 35–46.
[4] X. Han, G. Xu, M. Zhang, Z. Yang, Z. Yu, W. Huang, C. Meng, DE-GNN: Dual embedding with graph neural network for fine-grained encrypted traffic classification, Comput. Netw. 245 (2024) 110372.
[5] Z. Okonkwo, E. Foo, Z. Hou, Q. Li, Z. Jadidi, A graph representation framework for encrypted network traffic classification, Comput. Secur. 148 (2025) 104134, http://dx.doi.org/10.1016/j.cose.2024.104134.
[6] I. Akbari, M.A. Salahuddin, L. Aniva, N. Limam, R. Boutaba, B. Mathieu, S. Moteau, S. Tuffin, A look behind the curtain: Traffic classification in an increasingly encrypted web, Proc. ACM Meas. Anal. Comput. Syst. 5 (1) (2021) 1–26.
[7] V.F. Taylor, R. Spolaor, M. Conti, I. Martinovic, Robust smartphone app identification via encrypted network traffic analysis, IEEE Trans. Inf. Forensics Secur. 13 (1) (2018) 63–78.
[8] Z. Chen, G. Cheng, Z. Xu, S. Guo, Y. Zhou, Y. Zhao, Length matters: Scalable fast encrypted internet traffic service classification based on multiple protocol data unit length sequence with composite deep learning, Digit. Commun. Netw. 8 (3) (2022) 289–302.
[9] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, J. Yu, ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification, in: Proceedings of the ACM Web Conference 2022, 2022, pp. 633–642.
[10] Z. Wang, K.W. Fok, V.L. Thing, Machine learning for encrypted malicious traffic detection: Approaches, datasets and comparative study, Comput. Secur. 113 (2022) 102542.
[11] M. Shen, K. Ye, X. Liu, L. Zhu, J. Kang, S. Yu, Q. Li, K. Xu, Machine learning-powered encrypted network traffic analysis: A comprehensive survey, IEEE Commun. Surv. Tutor. 25 (1) (2023) 791–824.
[12] R. Ning, C. Xin, H. Wu, TrojanFlow: A neural backdoor attack to deep learning-based network traffic classifiers, in: IEEE INFOCOM 2022-IEEE Conference on Computer Communications, IEEE, 2022, pp. 1429–1438.
[13] B. Zhang, Z. Li, Z. Yang, X. He, M. Backes, M. Fritz, Y. Zhang, SecurityNet: Assessing machine learning vulnerabilities on public models, in: 33rd USENIX Security Symposium (USENIX Security 24), 2024, pp. 3873–3890.
[14] J.T. Holodnak, O. Brown, J. Matterer, A. Lemke, Backdoor poisoning of encrypted traffic classifiers, in: 2022 IEEE International Conference on Data Mining Workshops, ICDMW, IEEE, 2022, pp. 577–585.
[15] G. Severi, S. Boboila, A. Oprea, J. Holodnak, K. Kratkiewicz, J. Matterer, Poisoning network flow classifiers, in: Proceedings of the 39th Annual Computer Security Applications Conference, 2023, pp. 337–351.
[16] W. Wang, M. Zhu, J. Wang, X. Zeng, Z. Yang, End-to-end encrypted traffic classification with one-dimensional convolution neural networks, in: 2017 IEEE International Conference on Intelligence and Security Informatics, ISI, IEEE, 2017, pp. 43–48.
[17] C. Liu, L. He, G. Xiong, Z. Cao, Z. Li, FS-Net: A flow sequence network for encrypted traffic classification, in: IEEE INFOCOM 2019-IEEE Conference on Computer Communications, IEEE, 2019, pp. 1171–1179.
[18] W. Zheng, C. Gou, L. Yan, S. Mo, Learning to classify: A flow-based relation network for encrypted traffic classification, in: Proceedings of the Web Conference 2020, 2020, pp. 13–22.

[19] Z. Li, L. Bu, Y. Wang, Q. Ma, L. Tan, F. Bu, Hierarchical perception for encrypted traffic classification via class incremental learning, Comput. Secur. 149 (2025) 104195, http://dx.doi.org/10.1016/j.cose.2024.104195.
[20] M. Lotfollahi, M. Jafari Siavoshani, R. Shirali Hossein Zade, M. Saberian, Deep packet: A novel approach for encrypted traffic classification using deep learning, Soft Comput. 24 (3) (2020) 1999–2012.
[21] Y. Zhao, G. Dettori, M. Boffa, L. Vassio, M. Mellia, The sweet danger of sugar: Debunking representation learning for encrypted traffic classification, in: Proceedings of the ACM SIGCOMM 2025 Conference, 2025, pp. 296–310.
[22] R. Zhao, M. Zhan, X. Deng, Y. Wang, Y. Wang, G. Gui, Z. Xue, Yet another traffic classifier: A masked autoencoder based traffic transformer with multi-level flow representation, in: Proceedings of the AAAI Conference on Artificial Intelligence, 37, (4) 2023, pp. 5420–5427, http://dx.doi.org/10.1609/aaai.v37i4.25674.
[23] T. Wang, X. Xie, W. Wang, C. Wang, Y. Zhao, Y. Cui, NetMamba: Efficient network traffic classification via pre-training unidirectional mamba, in: 2024 IEEE 32nd International Conference on Network Protocols, ICNP, IEEE, 2024, pp. 1–11.
[24] G. Zhou, X. Guo, Z. Liu, T. Li, Q. Li, K. Xu, TrafficFormer: an efficient pre-trained model for traffic data, in: 2025 IEEE Symposium on Security and Privacy, SP, IEEE, 2025, pp. 1844–1860.
[25] W. Yang, W. Wu, F. Liu, L. Ma, Z. Zhang, B. Yang, PBA: Persistent backdoor attack on federated learning via distributed generative triggers, Comput. Netw. 277 (2026) 112038.
[26] R. Ding, L. Sun, W. Zang, L. Dai, Z. Ding, B. Xu, Towards universal and transferable adversarial attacks against network traffic classification, Comput. Netw. 254 (2024) 110790.
[27] J. Geng, J. Wang, Z. Fang, Y. Zhou, D. Wu, W. Ge, A survey of strategy-driven evasion methods for PE malware: Transformation, concealment, and attack, Comput. Secur. 137 (2024) 103595.
[28] B. Wang, Y. Yao, S. Shan, H. Li, B. Viswanath, H. Zheng, B.Y. Zhao, Neural Cleanse: Identifying and mitigating backdoor attacks in neural networks, in: 2019 IEEE Symposium on Security and Privacy, SP, IEEE, 2019, pp. 707–723.
[29] Y. Liu, W.-C. Lee, G. Tao, S. Ma, Y. Aafer, X. Zhang, ABS: Scanning neural networks for Back-doors by artificial brain stimulation, in: Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, 2019, pp. 1265–1282.
[30] W. Guo, L. Wang, X. Xing, M. Du, D. Song, TABOR: A highly accurate approach to inspecting and restoring trojan backdoors in AI systems, 2019, arXiv preprint arXiv:1908.01763.
[31] L. Zhu, R. Ning, C. Wang, C. Xin, H. Wu, GangSweep: Sweep out neural backdoors by GAN, in: Proceedings of the 28th ACM International Conference on Multimedia, 2020, pp. 3173–3181.
[32] H. Wang, Z. Xiang, D.J. Miller, G. Kesidis, MM-BD: Post-training detection of backdoor attacks with arbitrary backdoor pattern types using a maximum margin statistic, in: 2024 IEEE Symposium on Security and Privacy, SP, IEEE, 2024, pp. 1994–2012.
[33] S. Kolouri, A. Saha, H. Pirsiavash, H. Hoffmann, Universal litmus patterns: Revealing backdoor attacks in CNNs, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2020, pp. 301–310.
[34] Y. Gao, C. Xu, D. Wang, S. Chen, D.C. Ranasinghe, S. Nepal, STRIP: A defence against trojan attacks on deep neural networks, in: Proceedings of the 35th Annual Computer Security Applications Conference, 2019, pp. 113–125.
[35] W. Lyu, X. Lin, S. Zheng, L. Pang, H. Ling, S. Jha, C. Chen, Task-Agnostic detector for Insertion-Based backdoor attacks, in: Findings of the Association for Computational Linguistics: NAACL 2024, 2024, pp. 2808–2822, http://dx.doi.org/10.18653/v1/2024.findings-naacl.179.
[36] D. Tang, X. Wang, H. Tang, K. Zhang, Demon in the variant: Statistical analysis of DNNs for robust backdoor contamination detection, in: 30th USENIX Security Symposium, (USENIX Security 21), 2021, pp. 1541–1558.
[37] W. Ma, D. Wang, R. Sun, M. Xue, S. Wen, Y. Xiang, The “Beatrix” resurrections: Robust backdoor detection via gram matrices, in: Proceedings of the Network and Distributed System Security Symposium, NDSS, 2023, http://dx.doi.org/10.14722/ndss.2023.23069.
[38] X. Mo, Y. Zhang, L.Y. Zhang, W. Luo, N. Sun, S. Hu, S. Gao, Y. Xiang, Robust backdoor detection for deep learning via topological evolution dynamics, in: 2024 IEEE Symposium on Security and Privacy, SP, IEEE, 2024, pp. 2048–2066.
[39] S. Frolov, E. Wustrow, The use of TLS in censorship circumvention, in: NDSS, 2019.
[40] E. Rescorla, The Transport Layer Security (TLS) Protocol Version 1.3, RFC 8446, 2018, http://dx.doi.org/10.17487/RFC8446, URL https://www.rfc-editor.org/info/rfc8446.
[41] D. Benjamin, Applying Generate Random Extensions And Sustain Extensibility (GREASE) to TLS Extensibility, RFC 8701, 2020, http://dx.doi.org/10.17487/RFC8701, URL https://www.rfc-editor.org/info/rfc8701.
[42] D. Xue, M. Kallitsis, A. Houmansadr, R. Ensafi, Fingerprinting obfuscated proxy traffic with encapsulated TLS handshakes, in: 33rd USENIX Security Symposium (USENIX Security 24), 2024, pp. 2689–2706.
[43] J. Iyengar, M. Thomson, QUIC: A UDP-Based Multiplexed and Secure Transport, RFC 9000, 2021, http://dx.doi.org/10.17487/RFC9000, URL https://www.rfc-editor.org/info/rfc9000.
[44] T. Gu, B. Dolan-Gavitt, S. Garg, BadNets: Identifying vulnerabilities in the machine learning model supply chain, 2017, arXiv preprint arXiv:1708.06733.
[45] S.-M. Moosavi-Dezfooli, A. Fawzi, O. Fawzi, P. Frossard, Universal adversarial perturbations, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2017, pp. 1765–1773.
[46] Y. Mirsky, T. Doitshman, Y. Elovici, A. Shabtai, Kitsune: An ensemble of autoencoders for online network intrusion detection, in: Proceedings of the Network and Distributed System Security Symposium, NDSS, 2018, http://dx.doi.org/10.14722/ndss.2018.23204.
[47] X. Wu, X. Qiu, Z. Li, Y. Wang, J. Hu, C. Guo, H. Xiong, B. Yang, CATCH: Channel-Aware multivariate time series anomaly detection via frequency patching, in: International Conference on Learning Representations, ICLR, 2025.
[48] Y. Huang, X. Li, J. Geng, X. Zhu, TrafficT5: Multi-stage self-correcting framework for traffic generation, Comput. Netw. 274 (2026) 111858.



Shaotong Wang received the B.S. degree in Information and Computing Science from Chengdu University of Technology, China, in 2022. He is currently pursuing the M.S. degree in the College of Computer Science at Sichuan University, China. His research interests include encrypted traffic classification and deep learning security.



Jiaxuan Geng received the B.E. degree in Computer Science and Technology from Chongqing University of Posts and Telecommunications, China, in 2018 and the M.S. degree in Computer Science and Technology from Sichuan University, China, in 2021. He is currently pursuing the Ph.D. degree in the College of Computer Science at Sichuan University, China. His research interests include malware detection, software security and adversarial machine learning.



Yingjie Zhou received the Ph.D. degree from the School of Communication and Information Engineering, University of Electronic Science and Technology of China (UESTC), China, in 2013. He is currently a full professor with the College of Computer Science, Sichuan University (SCU), China. He was a visiting scholar with the Department of Electrical Engineering, Columbia University, New York. His current research interests include behavioral data analysis, network management, and time series analysis. He received the Best Paper Award at IEEE HPCC and the Best Demo Paper Award at IEEE ICME. He has served as the Senior Area Chair for IEEE ICASSP 2026, the Program Vice-Chair for IEEE HPCC 2022, the Local Arrangement Chair for IEEE BMSB 2021, etc. He has co-organized AI for Time Series (AI4TS) workshop at IJCAI since 2023. He is associate editors for IEEE Transactions on Emerging Topics in Computational Intelligence and Neural Networks.



Song Yang received the B.S. and M.S. degrees from Chongqing University of Posts and Telecommunications, Chongqing, in 1997 and 2007, respectively, and the Ph.D. degree from Beijing University of Posts and Telecommunications, Beijing, in 2014. From 2014 to 2019, he conducted postdoctoral research at the University of Electronic Science and Technology of China. He is currently Professorate Senior Engineer and Graduate Supervisor with the. His research interests include system architecture design and technological innovation of security products, the construction of R&D security engineering capability systems, and the design and establishment of security operations center systems.



Xing Yang currently serves as Professor and Ph.D. Supervisor at National University of Defense Technology (NUDT), and as Principal Scientist for AI Security at the Jianghuai Advance Technology Center. Between 2002 and 2012, he obtained his B.S., M.S., and Ph.D. degrees from the School of Electronic Engineering, specializing in optoelectronic sensing and AI security.



Dapeng Oliver Wu received the B.E. degree in electrical engineering from the Huazhong University of Science and Technology, Wuhan, China, in 1990, and the Ph.D. degree in electrical and computer engineering from Carnegie Mellon University, Pittsburgh, PA, USA, in 2003. He is currently the Yeung Kin Man Chair Professor of network science, City University of Hong Kong. His research interests include network science, communications, signal processing, artificial intelligence and information and network security. He has served as an Editor-in-Chief for IEEE Transactions on Network Science and Engineering; an Associate Editor for IEEE Transactions on Cloud Computing, IEEE Transactions on Computers, IEEE Transactions on Wireless Communications, and IEEE Transactions on Vehicular Technology; and a Guest Editor for IEEE Journal on Selected Areas in Communications. He is an IEEE Fellow.